



SIGGRAPH 2022
VANCOUVER+ 8-11 AUG

A Whirlwind Introduction to Computer Graphics

Mike Bailey

mjb@cs.oregonstate.edu

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the Owner/Author.

Copyright 2022 Mike Bailey

SIGGRAPH '22 Educator's Forum, August 07-11, 2022, Vancouver, BC, Canada

ACM 978-1-4503-9366-9/22/08.

10.1145/3532724.3535605



<http://cs.oregonstate.edu/~mjb/whirlwind>

mjb - June 3, 2022



Mike Bailey
Oregon State University

mjb@cs.oregonstate.edu



This work is licensed under a [Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License](https://creativecommons.org/licenses/by-nc-nd/4.0/)

**A Whirlwind
Introduction to
Computer Graphics**

<http://cs.oregonstate.edu/~mjb/whirlwind>

mjb - June 3, 2022

Course Goals

- Provide a background for the amazing things you will hear about in the other SIGGRAPH 2022 venues
- Create an understanding of common computer graphics vocabulary
- Help you understand the significance of the images and animations that you will see
- Provide references for further study

<http://cs.oregonstate.edu/~mjb/whirlwind>



4

Mike Bailey

- Professor of Computer Science, Oregon State University
- Has been in computer graphics for over 30 years
- Has had over 10,000 students in his university classes
- mjb@cs.oregonstate.edu

Welcome! I'm
happy to be here.
I hope you are too!



<http://cs.oregonstate.edu/~mjb/whirlwind>

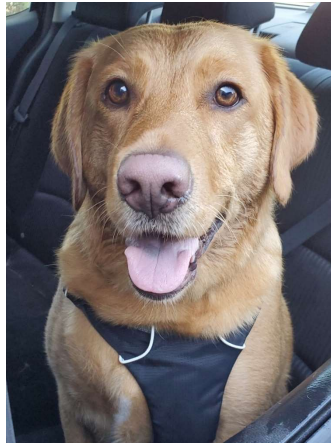
mjb - June 3, 2022

I am Recording this at Home:

5

I Apologize in Advance for What You Might Hear in the Background ☺

We sometimes bark
at nothing – it's not
Mike's fault!

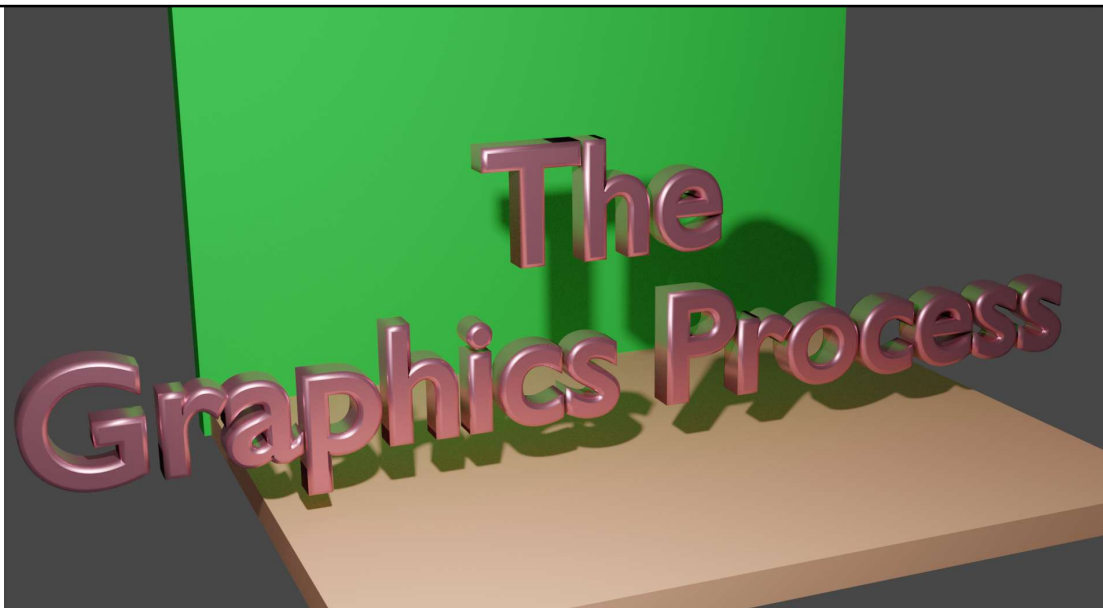


Sections

1. How the computer graphics pieces fit together
2. Modeling
3. Rendering
4. Animation
5. Finding More Information

<http://cs.oregonstate.edu/~mjb/whirlwind>

7

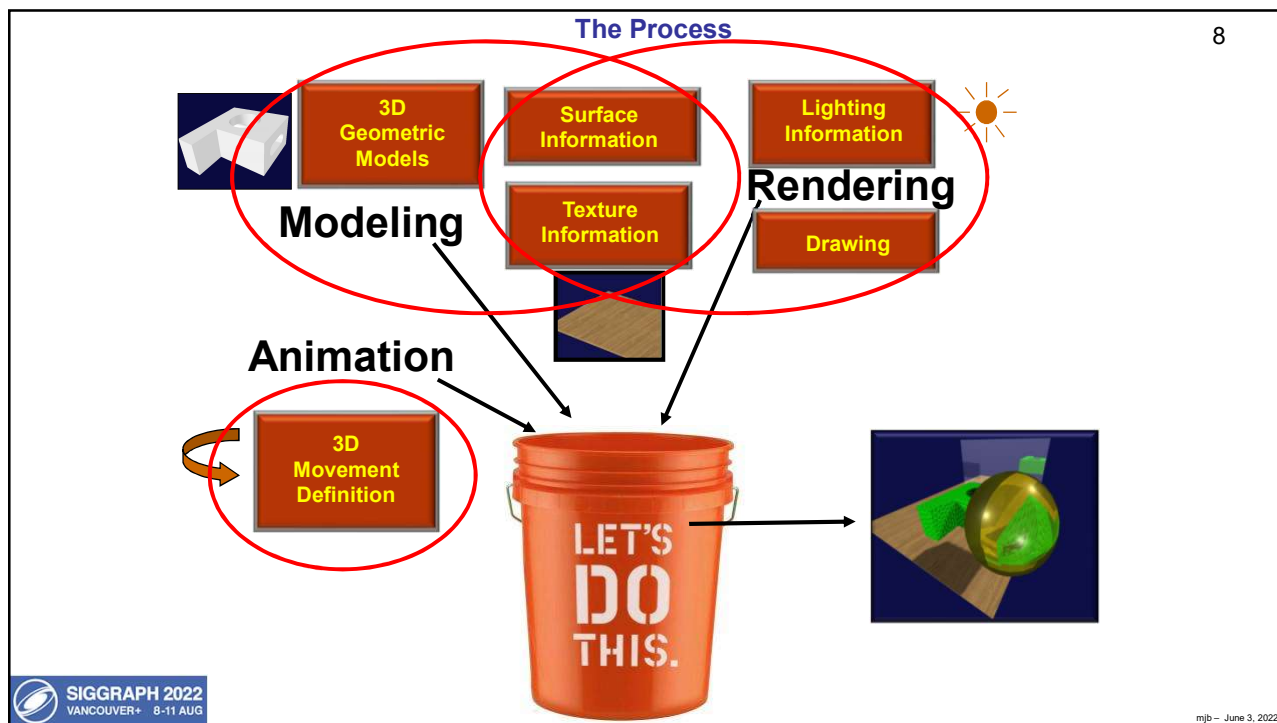


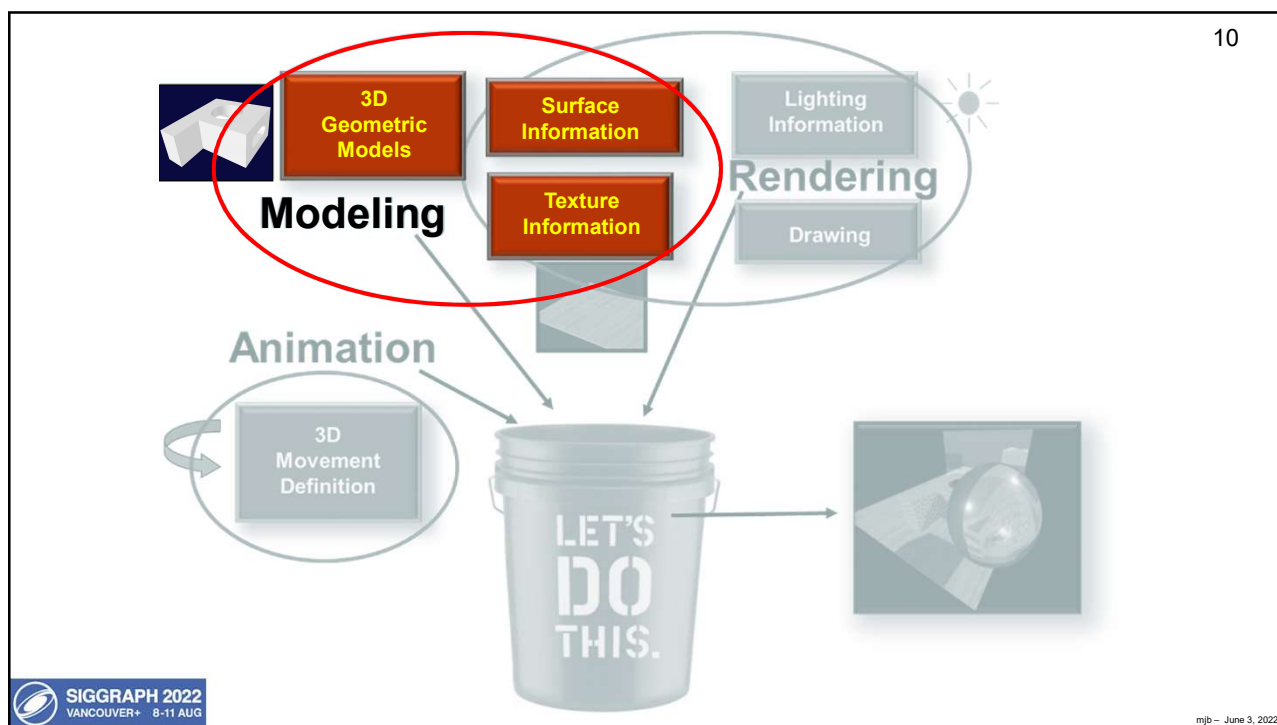
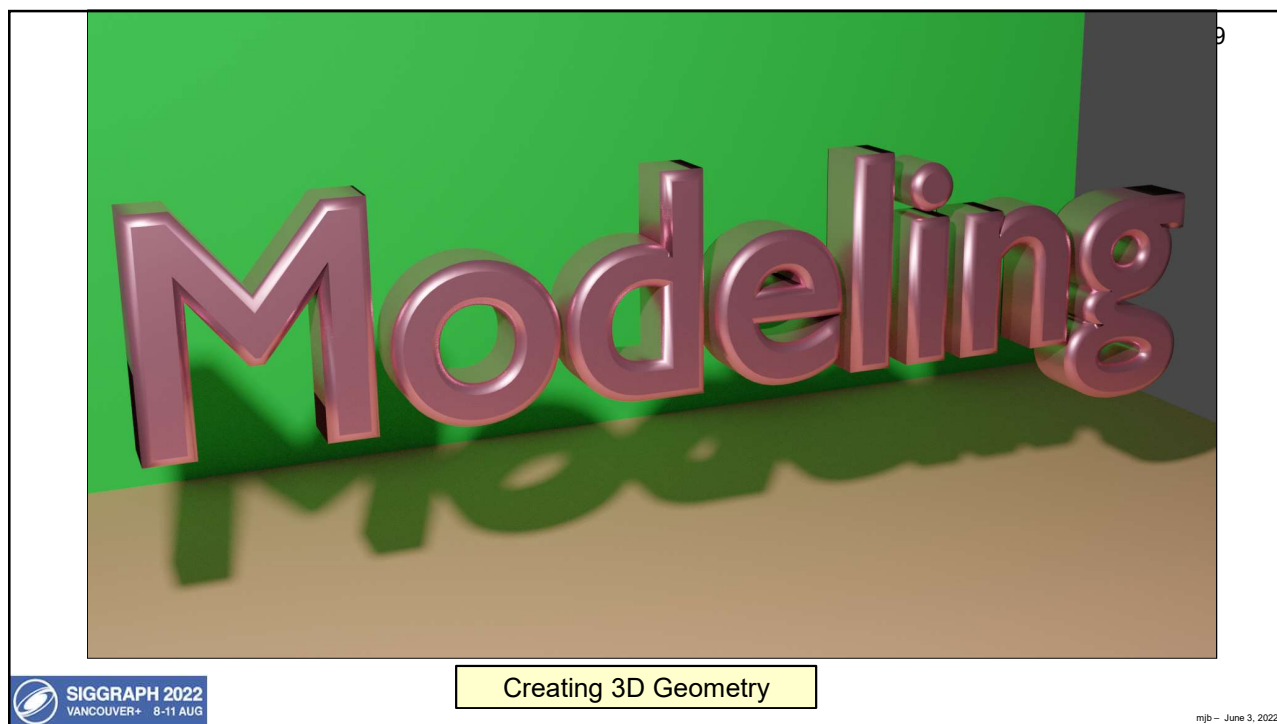
The Graphics Process

What are all the pieces that go into making the graphics you will be see?
What does it take to make them?



mjb - June 3, 2022



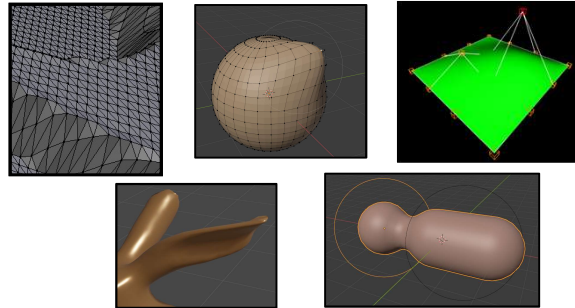


What do we mean by “Modeling”?

11

In computer graphics applications, how we model geometry depends on what we would like to use the geometry for:

- Looking at its appearance
- Interacting with its shape?
- How does it interact with its environment?
- What is its surface area and volume?
- Will it be able to be 3D-printed?
- Etc.



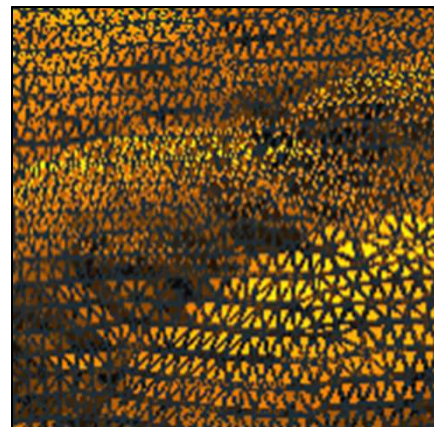
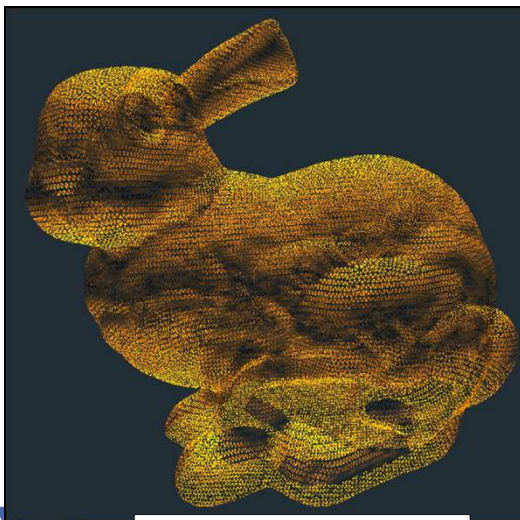
Want to experiment with some free modeling programs?
Want some notes on how to get started?

<http://cs.oregonstate.edu/~mjb/blender>
<http://cs.oregonstate.edu/~mjb/sketchup>
<http://cs.oregonstate.edu/~mjb/tinkercad>

Explicitly Listing Geometry and Topology

12

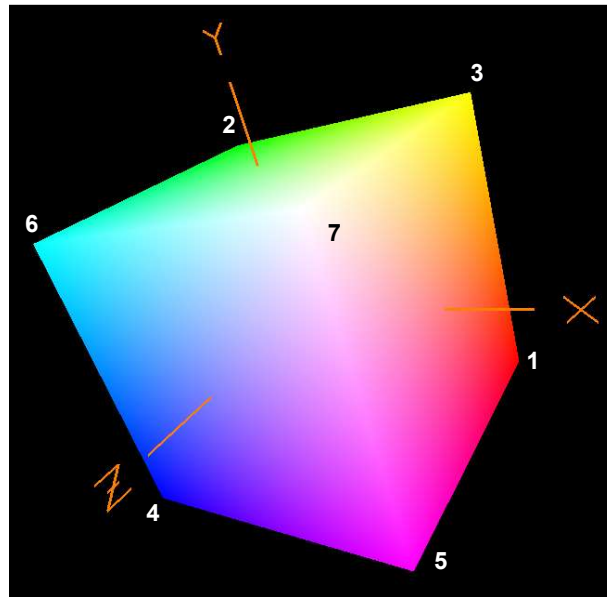
Models defined this way can consist of thousands of vertices and faces – we need some way to describe them effectively



This is often called a **Mesh**.

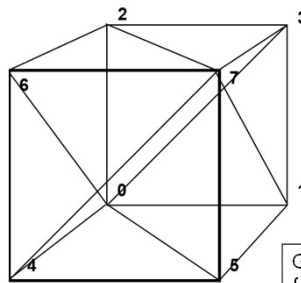
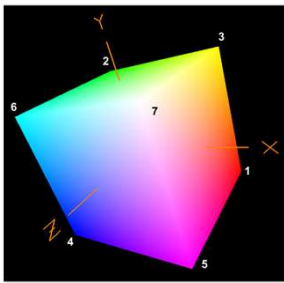
Cube Mesh Example

13



Explicitly Listing Geometry and Topology – Triangular Meshes

14



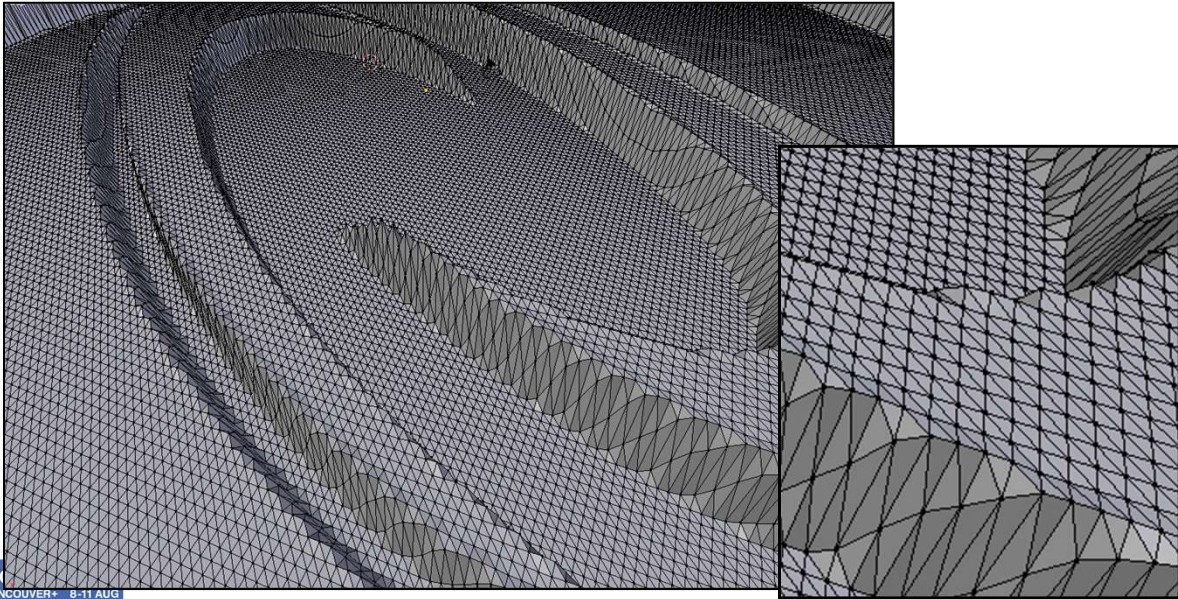
```
static GLfloat CubeVertices[ ][3] =
{
    { -1., -1., -1. },
    { 1., -1., -1. },
    { -1., 1., -1. },
    { 1., 1., -1. },
    { -1., -1., 1. },
    { 1., -1., 1. },
    { -1., 1., 1. },
    { 1., 1., 1. }
};
```

```
static GLfloat CubeColors[ ][3] =
{
    { 0., 0., 0. },
    { 1., 0., 0. },
    { 0., 1., 0. },
    { 1., 1., 0. },
    { 0., 0., 1. },
    { 1., 0., 1. },
    { 0., 1., 1. },
    { 1., 1., 1. }
};
```

```
GLuint TriangleCubeIndices[ ][3] =
{
    { 0, 2, 3 },
    { 0, 3, 1 },
    { 4, 5, 7 },
    { 4, 7, 6 },
    { 1, 3, 7 },
    { 1, 7, 5 },
    { 0, 4, 6 },
    { 0, 6, 2 },
    { 2, 6, 7 },
    { 2, 7, 3 },
    { 0, 1, 5 },
    { 0, 5, 4 }
};
```

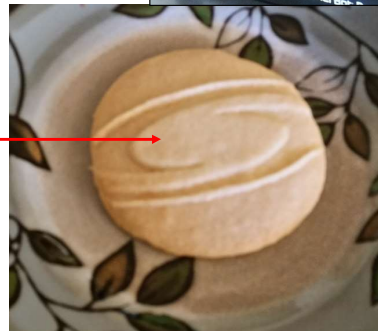
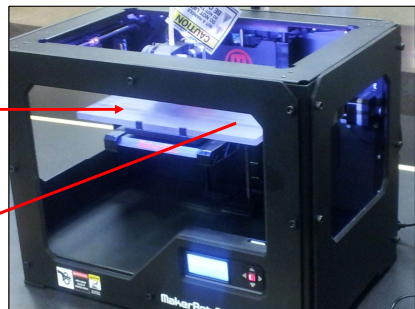
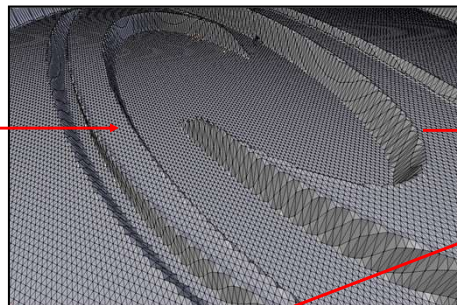
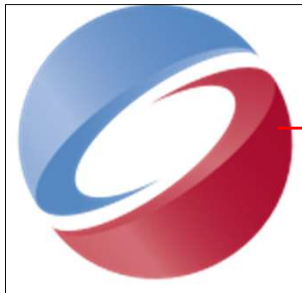
Triangular Meshes are Extra Important These Days Because
3D Printing Requires a Triangular Mesh Data Format

15



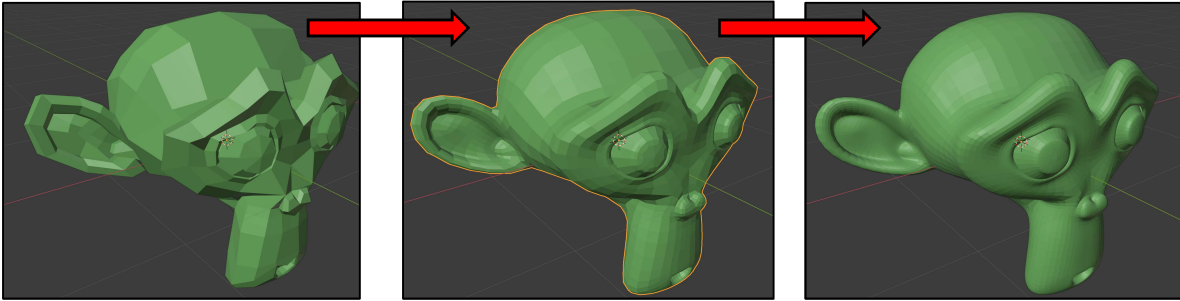
3D geometric modeling at its very best -- mmmm... :-)

16



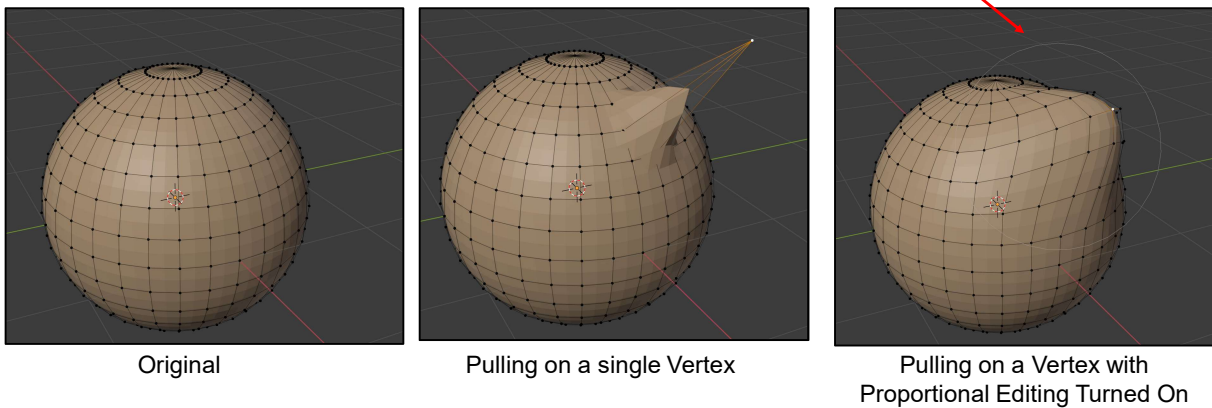
Meshes Can Be Smoothed

17



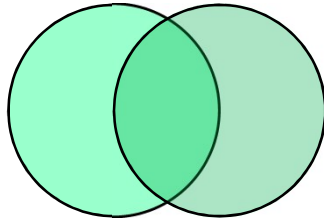
Meshes Can Be Edited

18

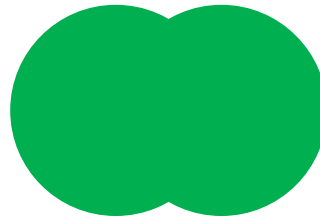


Another way to Model:
Remember Venn Diagrams (2D Boolean Operators)?

19



Two Overlapping Shapes



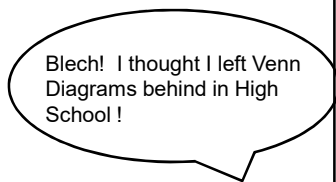
Union



Intersection

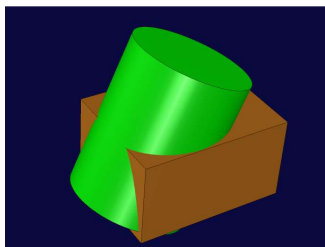


Difference

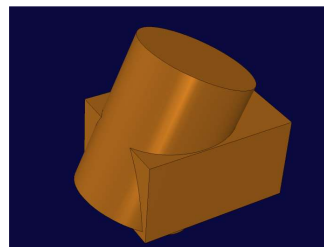


Solid Modeling Using 3D Boolean Operators

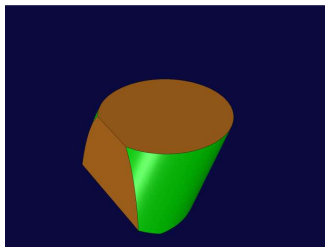
20



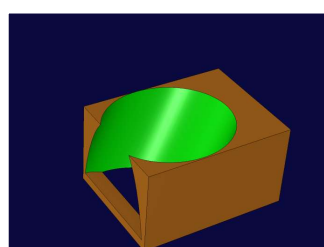
Two Overlapping Solids



Union



Intersection

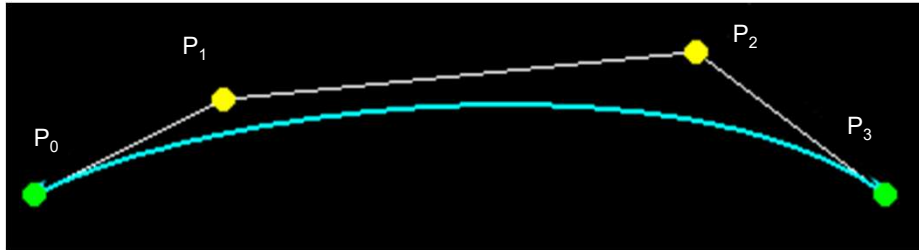


Difference

This is often called **Constructive Solid Geometry**, or **CSG**

Another way to Model: Curve Sculpting – Bézier Curve Sculpting

21

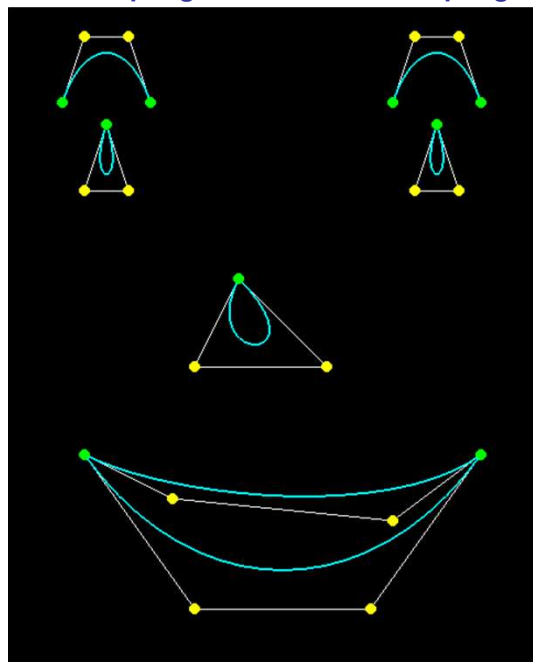


$$P(t) = (1-t)^3 P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t)P_2 + t^3 P_3$$

$$0 \leq t \leq 1.$$

Curve Sculpting – Bézier Curve Sculpting Example

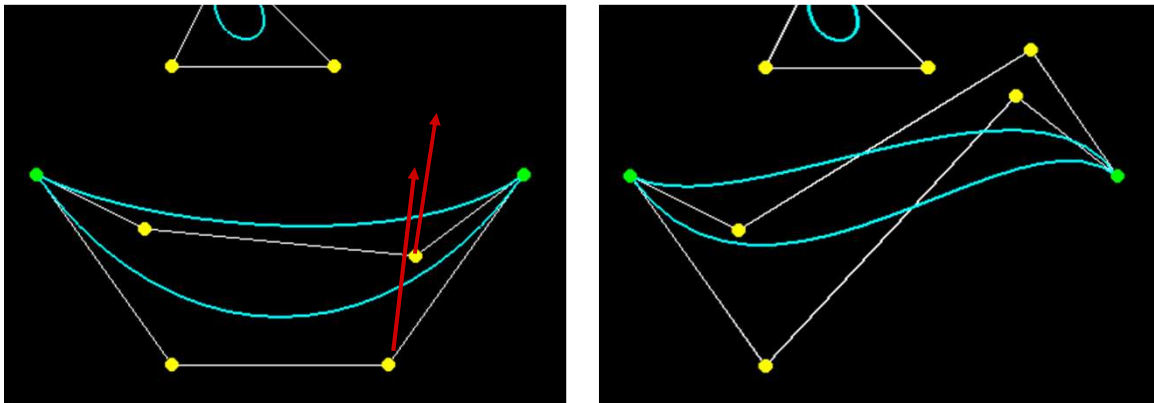
22



curves.mp4

Curve Sculpting – Bézier Curve Sculpting Example

23

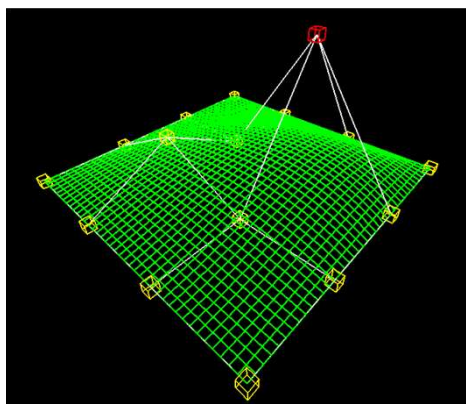


A *Small* Amount of Input Change Results in a *Large* Amount of Output Change

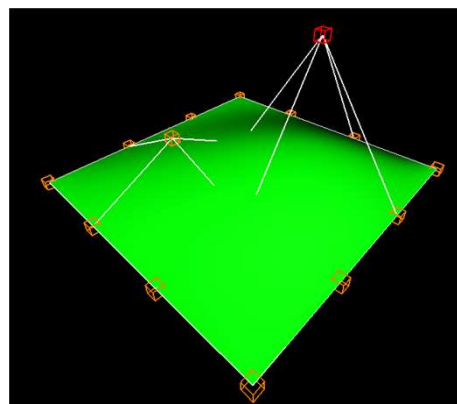
Another way to Model: Surface Sculpting

24

In general, these are referred to as **Patches**. These, in particular, are Beziér patches.
Non-uniform Rational B-spline Surfaces, or NURBS, are another popular type.



Wireframe

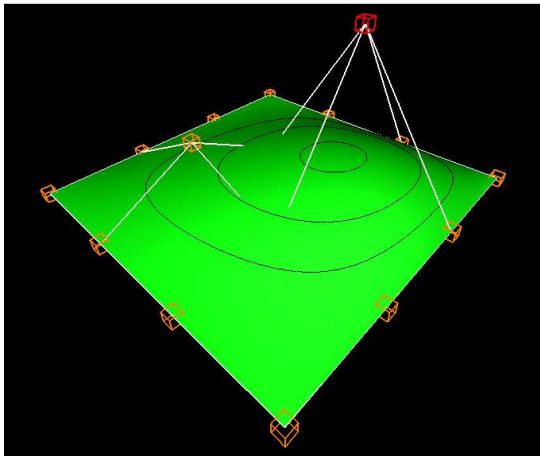


Surface

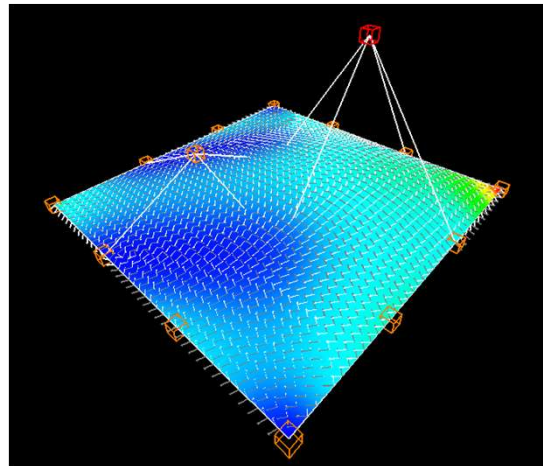
Like the curve sculpting, a *Small* Amount of Input Change Results in a *Large* Amount of Output Change

Surface Equations can also be used for Analysis

25



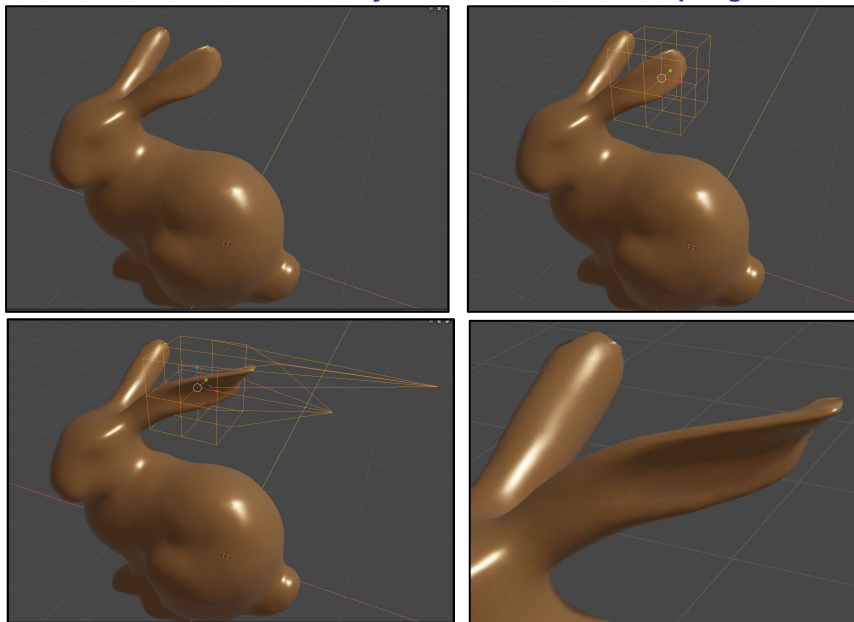
Showing Contour Lines



Showing Curvature

Another Way to Model: Volume Sculpting

26



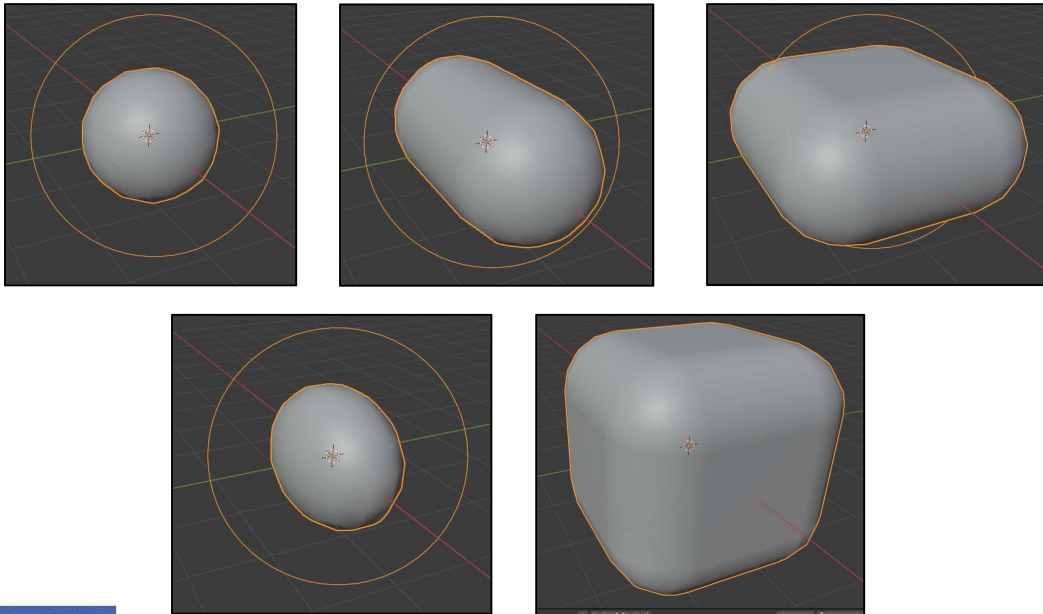
This is often called a “Lattice” or a “Cage”.



lattice.mp4

Metaball Objects

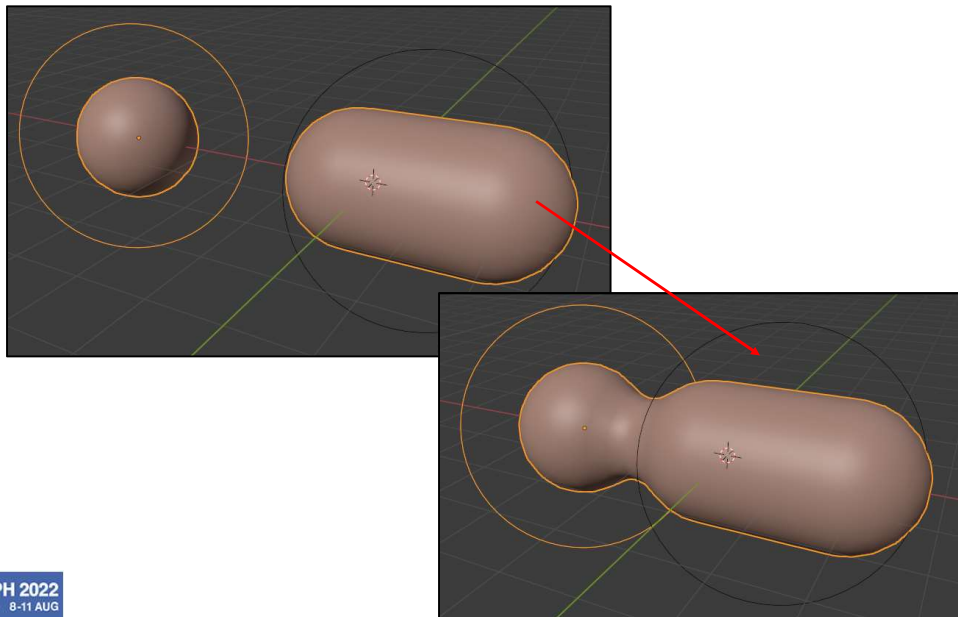
27



Metaball Objects

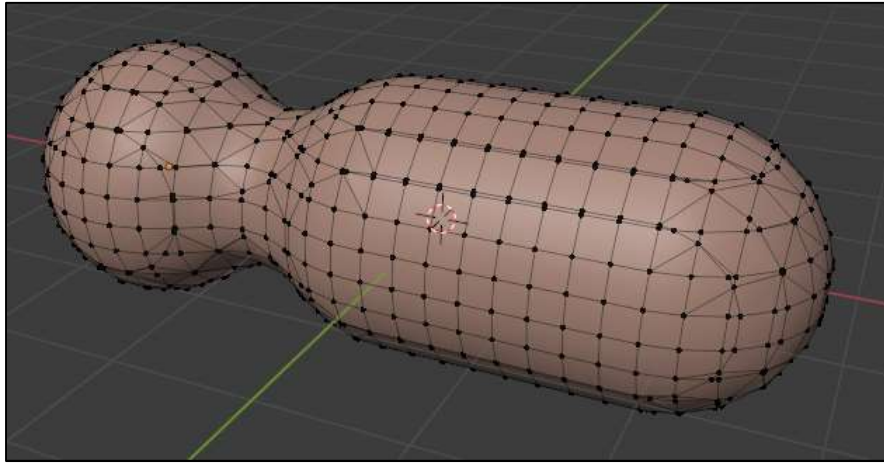
28

The cool thing is that, if you move them close enough together, they will “glom” into a single object



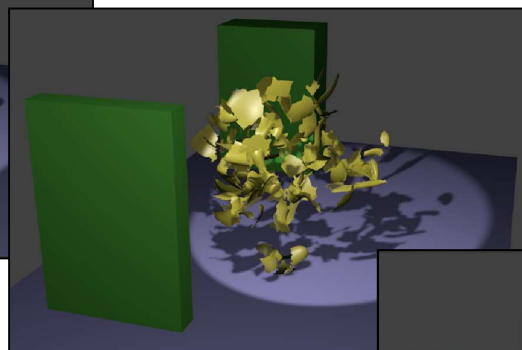
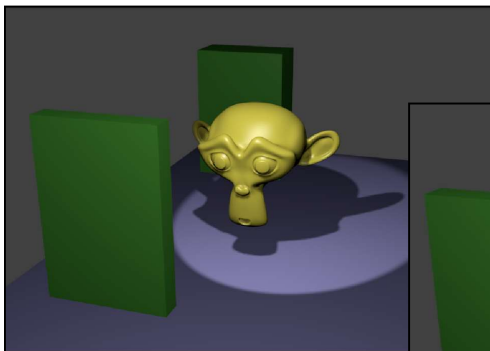
Metaball Objects Can Be Turned into Meshes for Later Editing

29



Geometric Models can also be used for Physical Simulation

30



Blender's *Explosion* feature

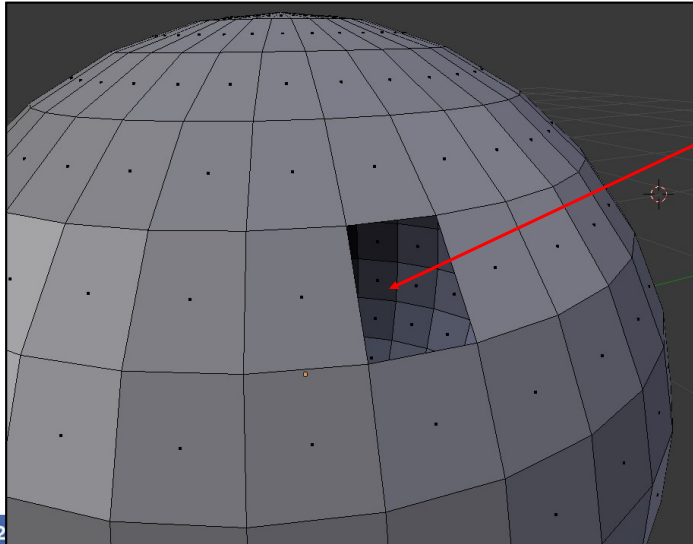


Blender's *Smoke* feature

Object Modeling Rules for 3D Printing

31

The object must be a legal solid. It must have a definite inside and a definite outside. It can't have any missing face pieces.

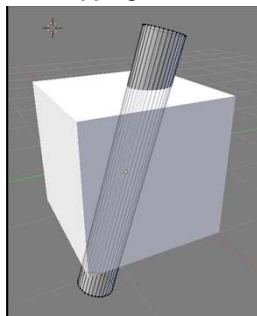


Object Modeling Rules for 3D Printing

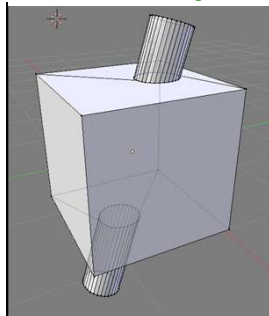
32

Objects cannot pass through other objects. If you want two shapes together, do a CSG union on them so that they become one complete object.

Overlapping in 3D -- **bad**



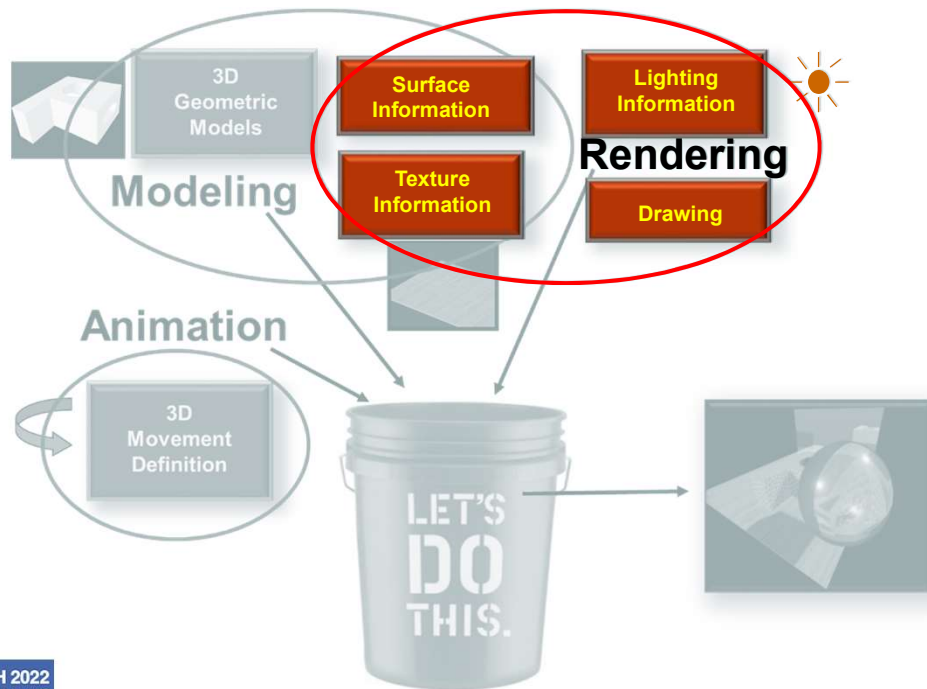
Boolean union -- **good**





33

Creating an image

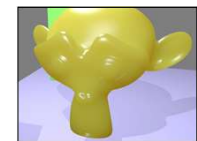
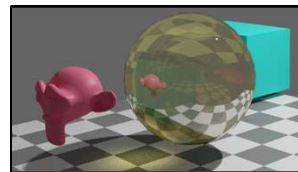
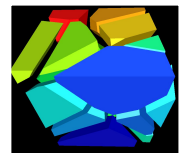
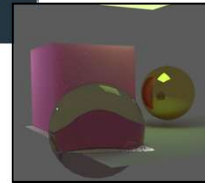


34

Rendering

Rendering is the process of creating an image of geometric models. Again, there are questions you need to ask first:

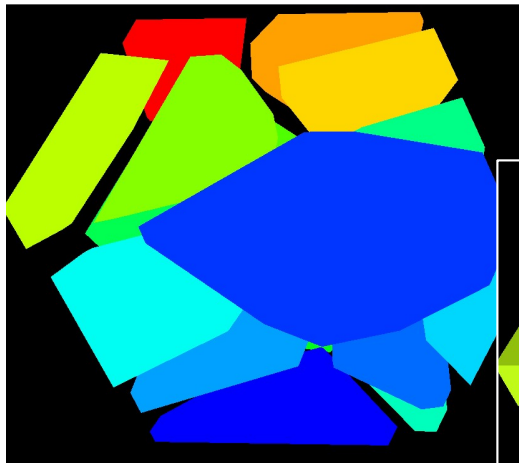
- Why am I doing this?
- How realistic do I want this image to be?
- How much compute time do I want this to take?
- Do I need to take lighting into account?
- Does the illumination need to be global or will local do?
- Do I need to create shadows?
- Do I need to create reflections and refractions?



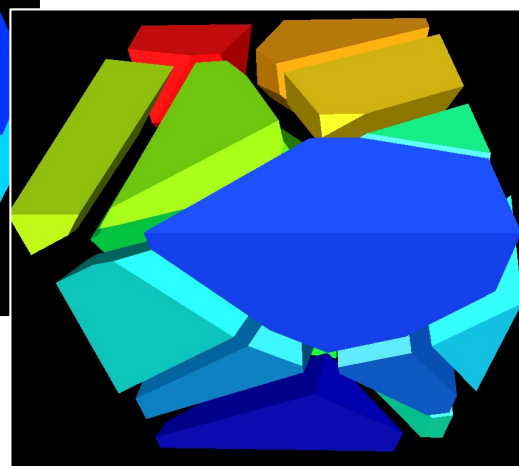
Want to experiment with a free rendering program?
Want some notes on how to get started?
<http://cs.oregonstate.edu/~mjb/blender>

Why Do We Care About Lighting?

No lighting

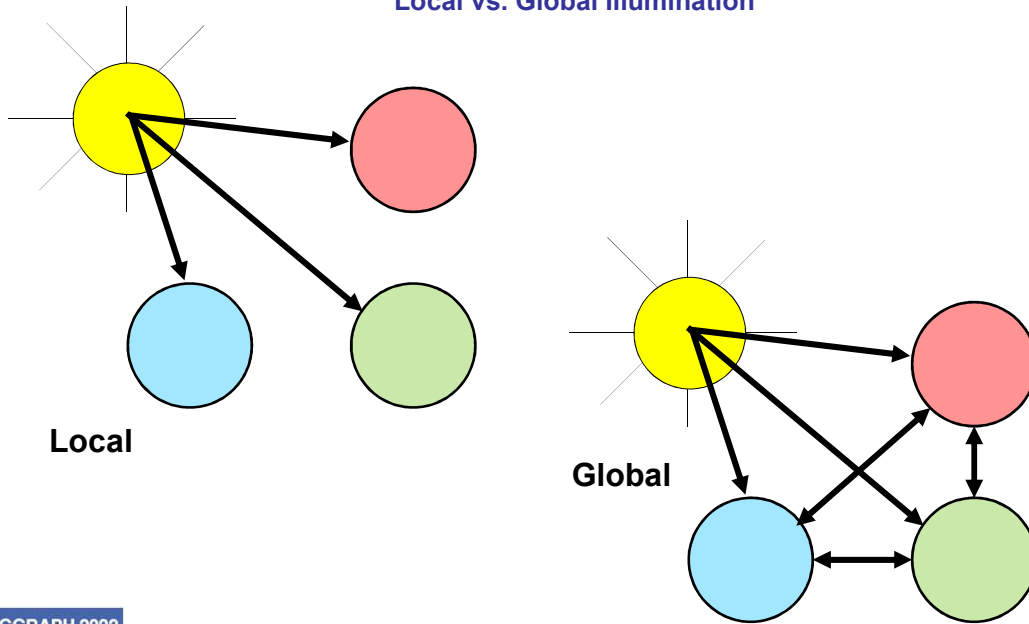


Lighting

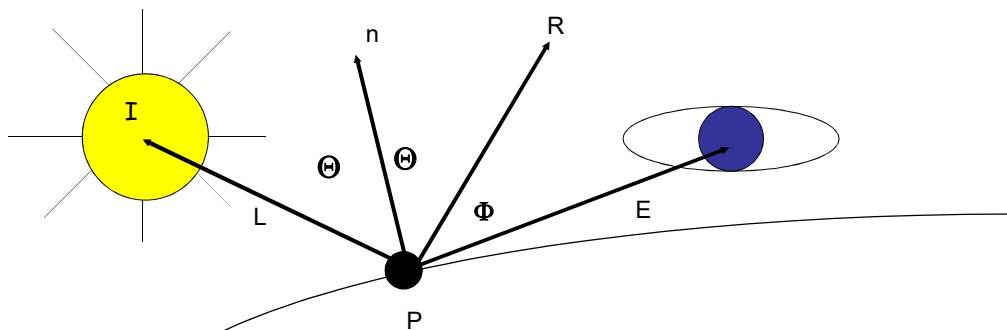


Lighting makes it possible to tell the difference between surfaces or parts of surfaces

Local vs. Global Illumination



A Common type of Local Illumination: Ambient-Diffuse-Specular (ADS)



P	Point being illuminated
I	Light intensity
L	Unit vector from point to light
n	Unit vector surface normal
R	Perfect reflection unit vector
E	Unit vector to eye position

A Common type of Local Illumination: Ambient-Diffuse-Specular (ADS)

39

1. Ambient = a constant Accounts for light bouncing “everywhere”
2. Diffuse = $I \cdot \cos\Theta$ Accounts for the angle between the incoming light and the surface normal
3. Specular = $I \cdot \cos^S\phi$ Accounts for the angle between the “perfect reflector” and the eye; also the exponent, S, accounts for surface shininess

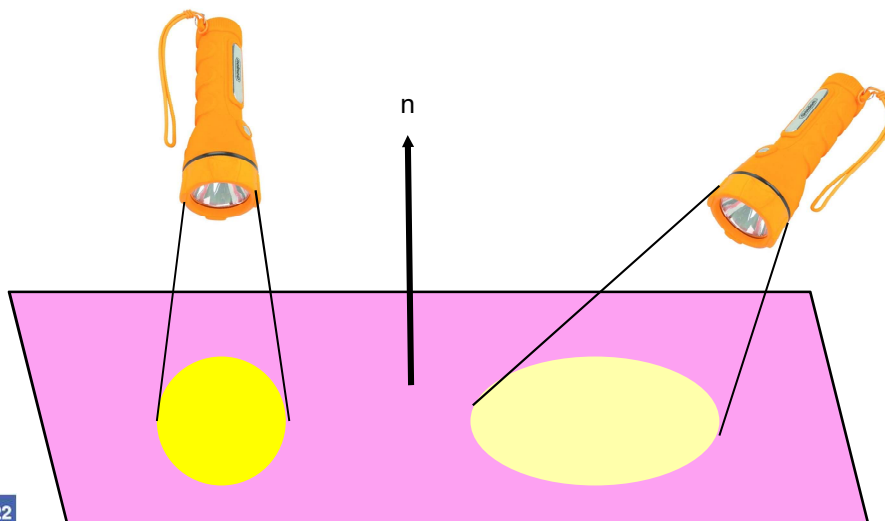
Note that $\cos\Theta$ is just the dot product between unit vectors L and n

Note that $\cos\phi$ is just the dot product between unit vectors R and E

Diffuse Lighting works because of spreading out the same amount of light energy across more surface area

40

$$\text{Diffuse} = I \cdot \cos\Theta$$



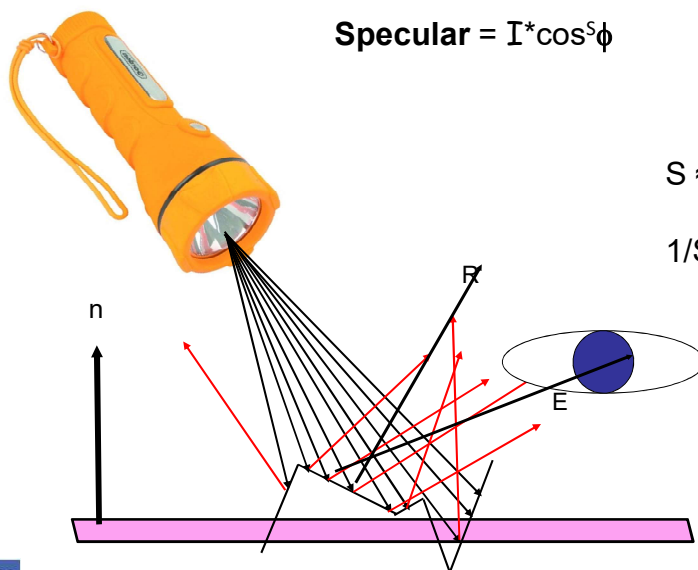
The Specular Lighting equation is a heuristic that approximates reflection from a rough surface

41

$$\text{Specular} = I * \cos^S \phi$$

$S \approx$ "shininess"

$1/S \approx$ "roughness"



Put them all together!

42



+



+



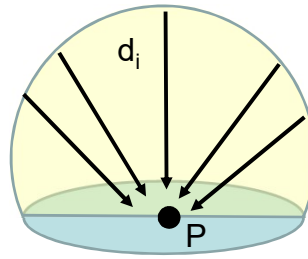
=



lighting.mp4

Global Illumination: The Rendering Equation

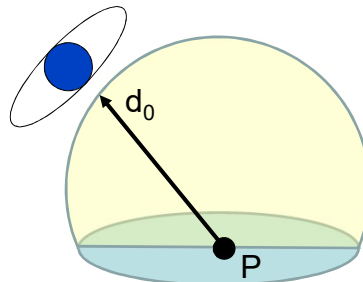
43



Ω

Light arriving at Point P from everywhere

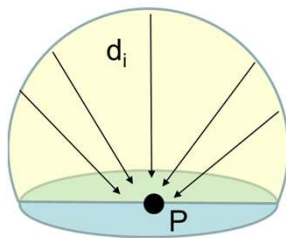
$$B(P, d_0, \lambda) = E(P, d_0, \lambda) + \int_{\Omega} B(P, d_i, \lambda) f(\lambda, d_i, d_0) (d_i \cdot \hat{n}) d\Omega$$



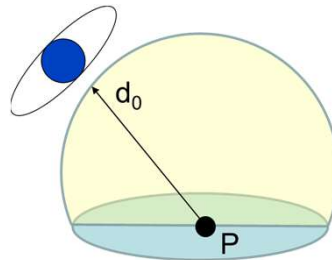
Light departing from Point P in the direction that we are viewing the scene from

The Rendering Equation

44



Ω

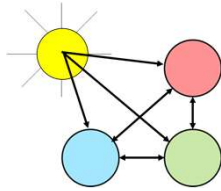


$$B(P, d_0, \lambda) = E(P, d_0, \lambda) + \int_{\Omega} B(P, d_i, \lambda) f(\lambda, d_i, d_0) (d_i \cdot \hat{n}) d\Omega$$

In plain language, this is a light balance equation:

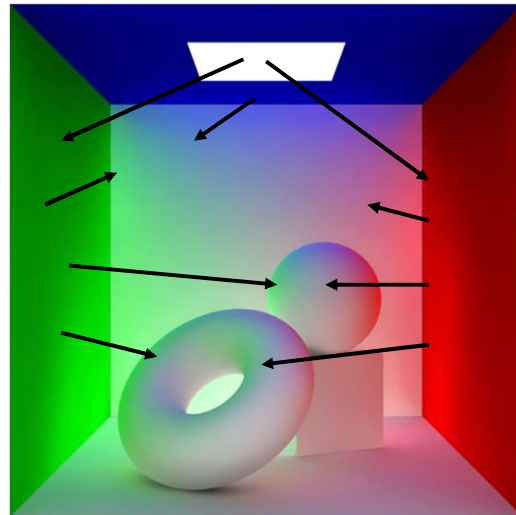
“The light shining from the point P towards your eye is the reflection of the incoming light directed to the point P from all of the other points in the scene.”

The Lighting Equation at Work



- The left wall is green.
- The right wall is red.
- The back wall is white.
- The ceiling is blue with a light source in the middle of it.
- The objects sitting on the floor are white.

If the appearance of an object is also affected by the appearances of other objects, then you have **Global Illumination**.



<http://www.swardson.com/unm/tutorials/mentalRay3/>

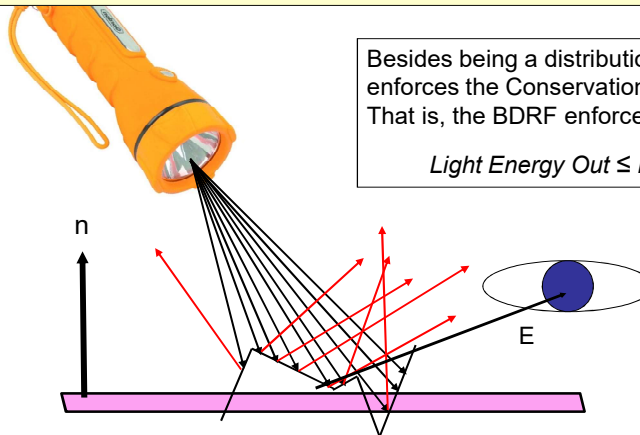
When light hits a surface, it bounces in particular ways depending on the angle and the material

This distribution of bounced light rays is called the **Bidirectional Reflectance Distribution Function, or BRDF**.

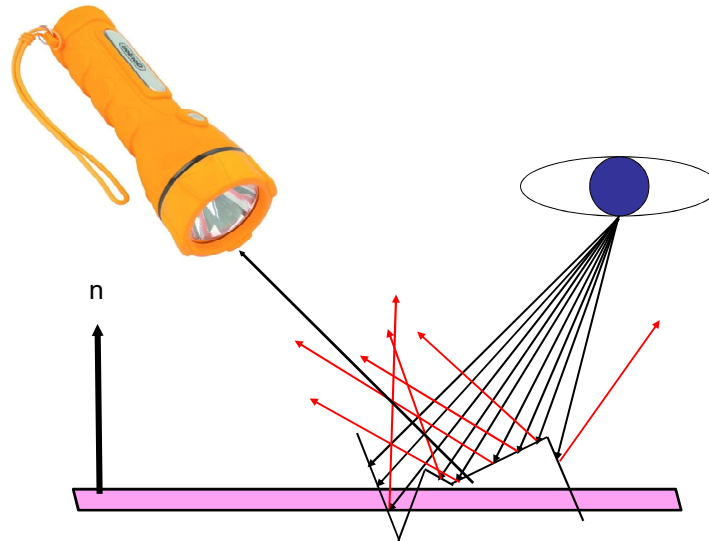
For a given material, the BRDF behavior of a light ray is a function of 4 variables: the 2 spherical coordinates of the incoming ray and the 2 spherical coordinates of the outgoing ray.

Besides being a distribution, the BRDF enforces the Conservation of Energy law. That is, the BRDF enforces

$$\text{Light Energy Out} \leq \text{Light Energy In}$$

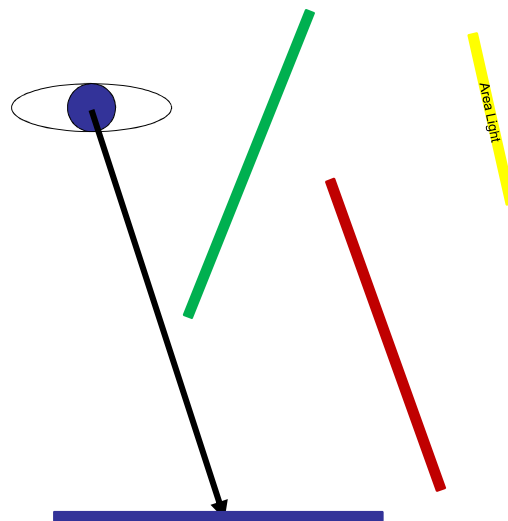


Usually it is easier to trace from the eye



Physically-Based Rendering

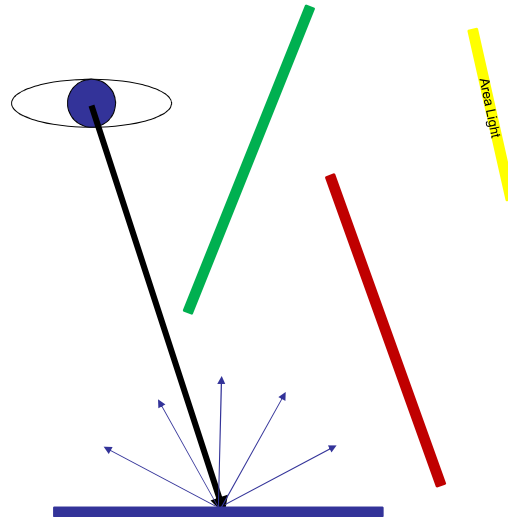
Let light can bounce around the scene, depending on how the different materials behave.



Physically-Based Rendering

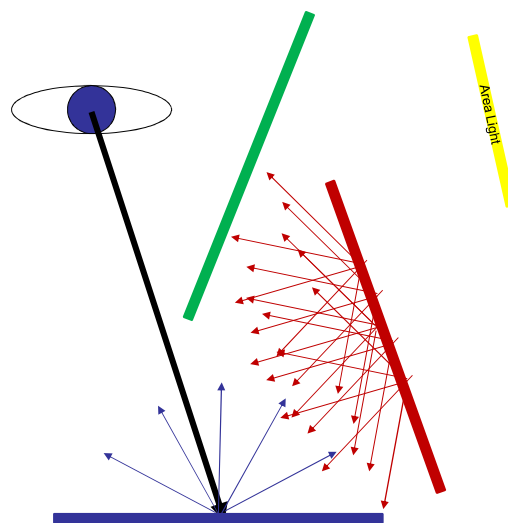
49

Let light can bounce around the scene, depending on how the different materials behave.



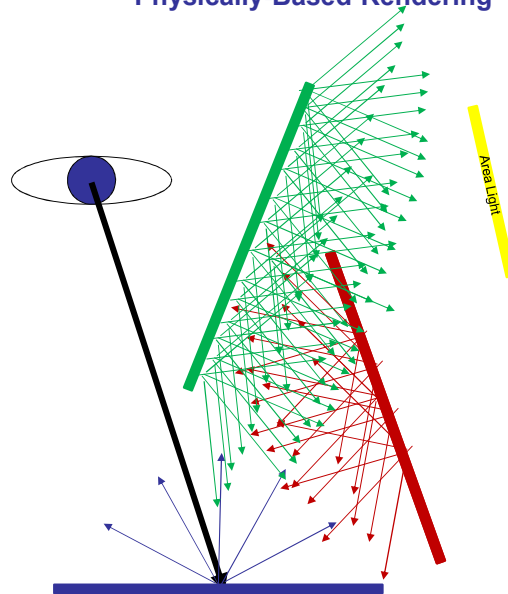
Physically-Based Rendering

50



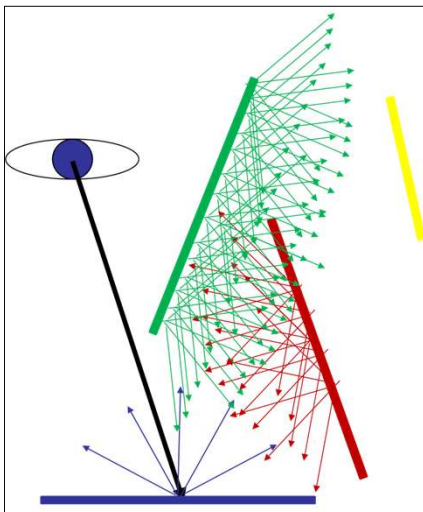
Physically-Based Rendering

51



Physically-Based Rendering

52



Clearly this is capable of spawning an infinite number of rays. How do we handle this?

For a small-ish number of bounces, we can evenly distribute a collection of rays.

For lots of bounces, it's **Monte Carlo** simulation to the rescue!

$$LightGathered = \frac{\sum_{i=0}^{N-1} ResultOfRaysCastInRandomDirection}{N}$$

Recurse by applying this equation for all ray hits (yikes!)

Physically-Based Rendering using the Blender Cycles Renderer

53

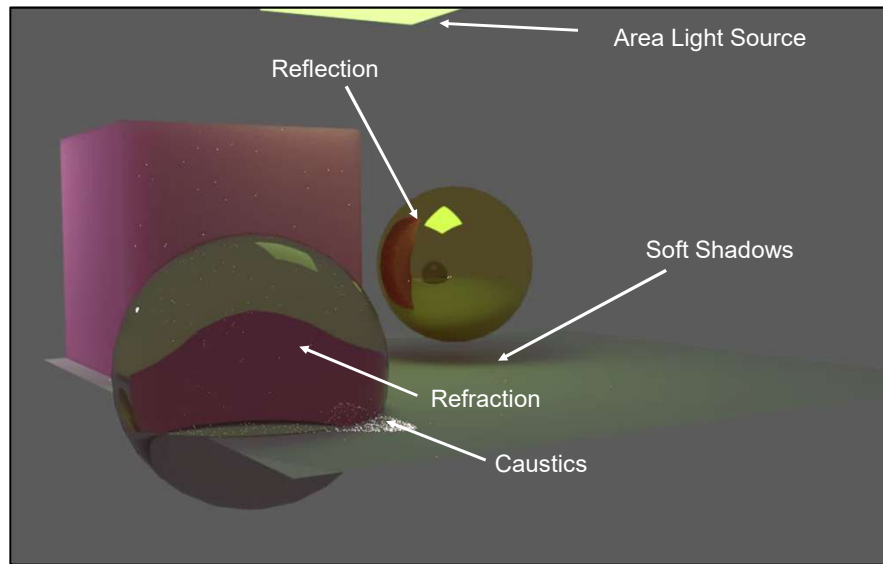


Image: Mike Bailey

Physically-Based Rendering using the Blender Cycles Renderer

54

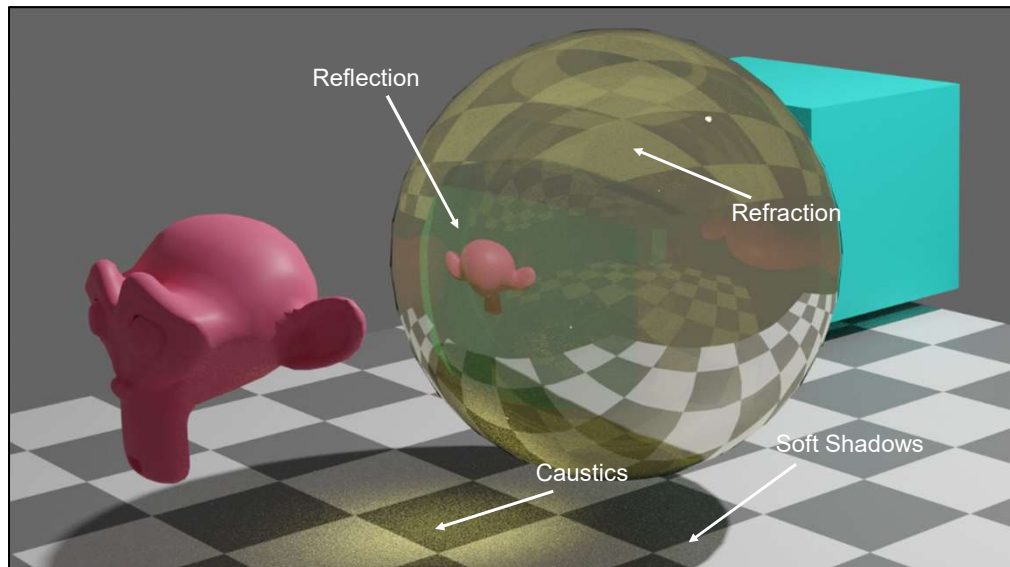
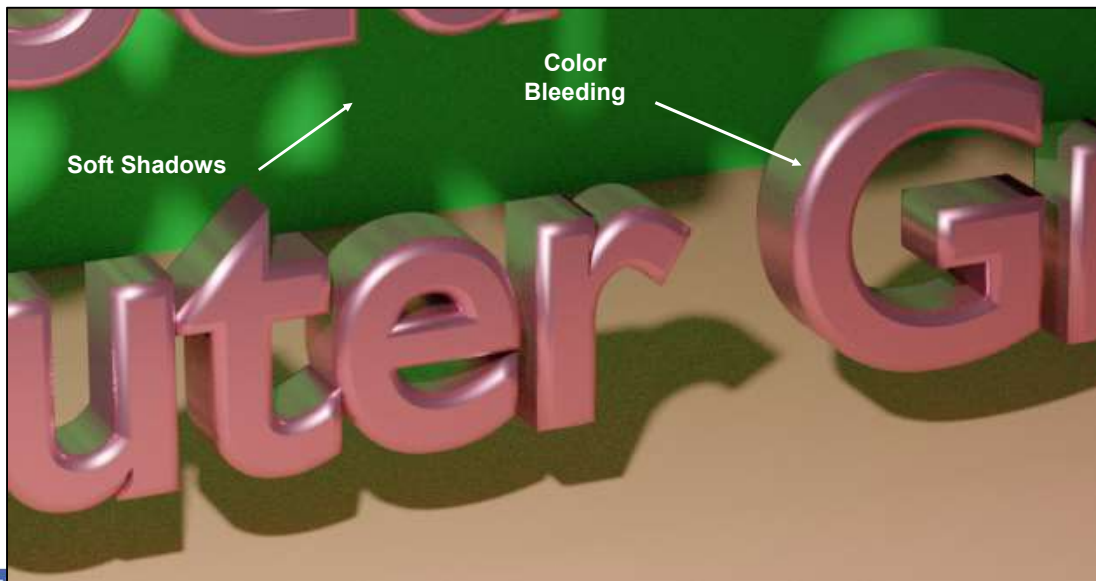


Image: Mike Bailey

An Example from the Title Slide

55

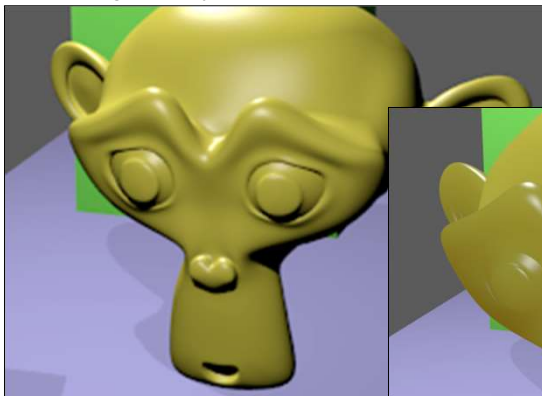


Subsurface Scattering

56

Subsurface Scattering models light bouncing around *within* an object before coming back out.

This is a good way to represent skin, wax, milk, paraffin, etc.



Without Subsurface Scattering



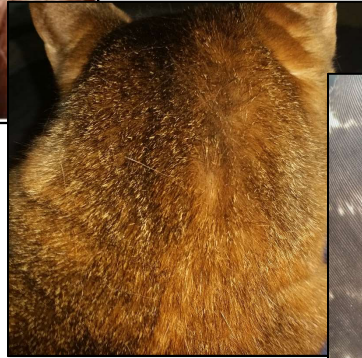
With Subsurface Scattering

Tricky Lighting Situations

57



Hair



Fur

Feathers



Watch for these at the conference and in CG movies!



mjb - June 3, 2022

An Neat Global Illumination-ish Trick: Screen Space Ambient Occlusion (SSAO)

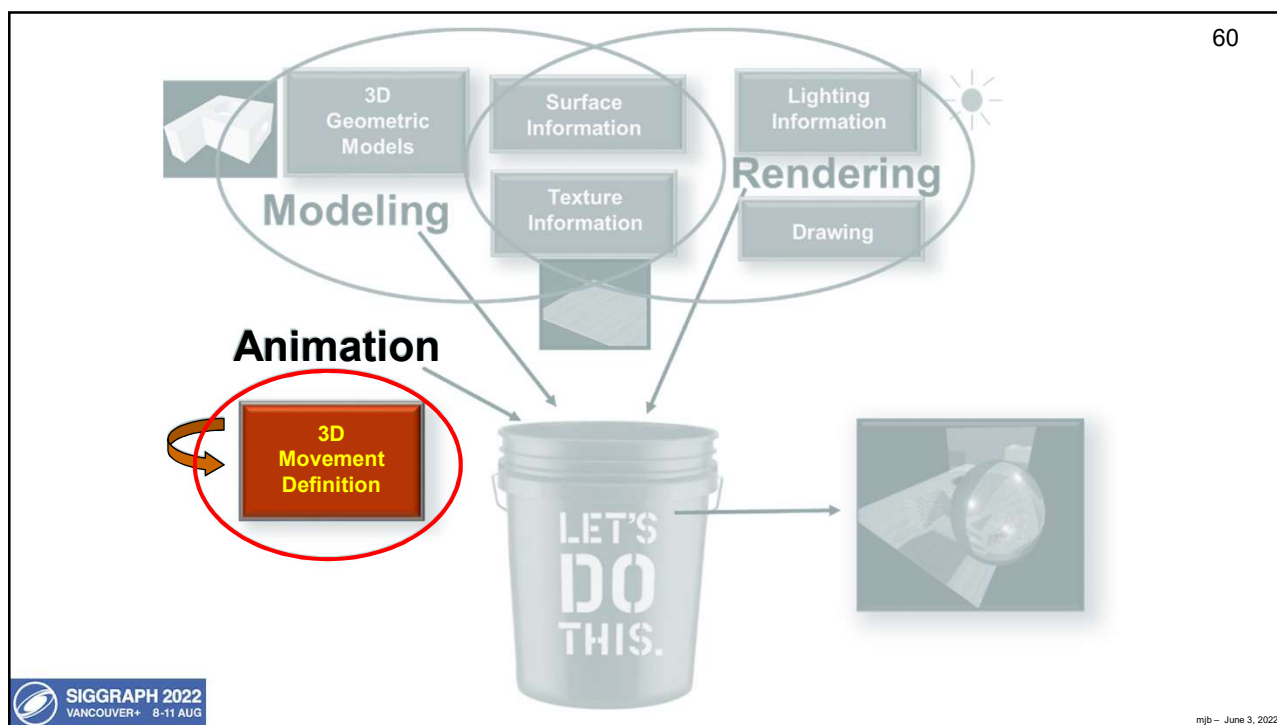
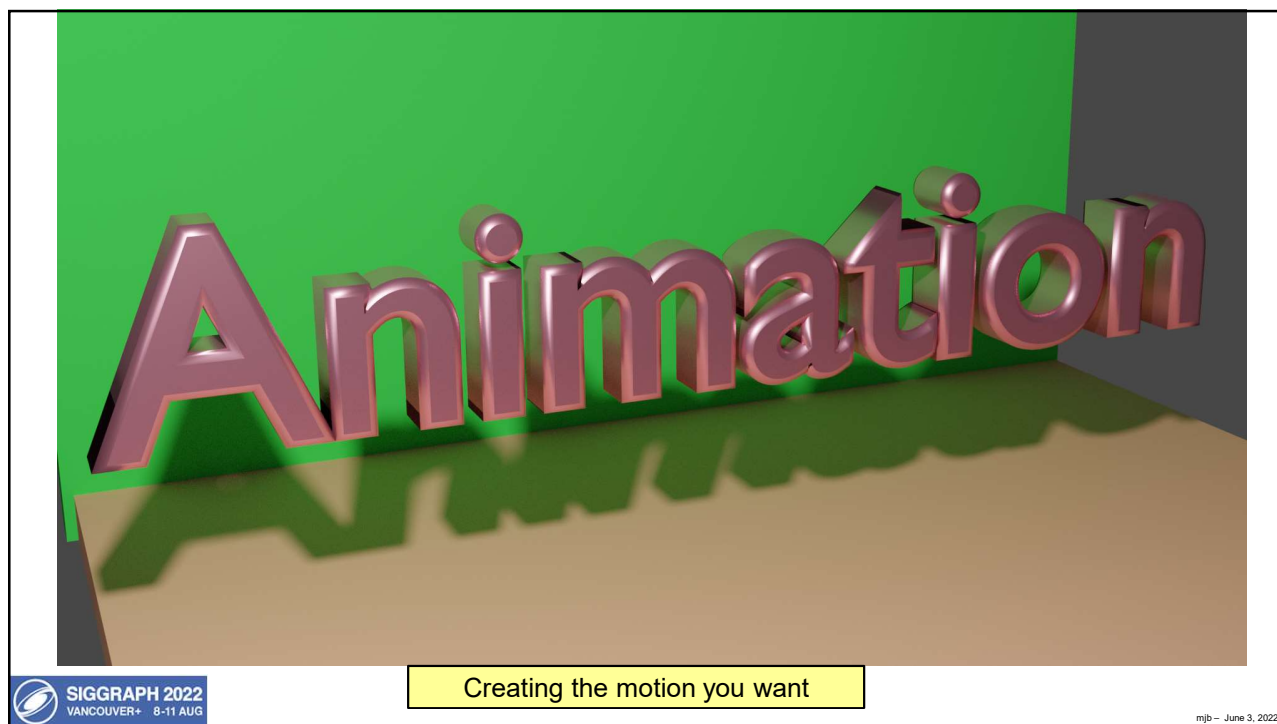
58



Kitware



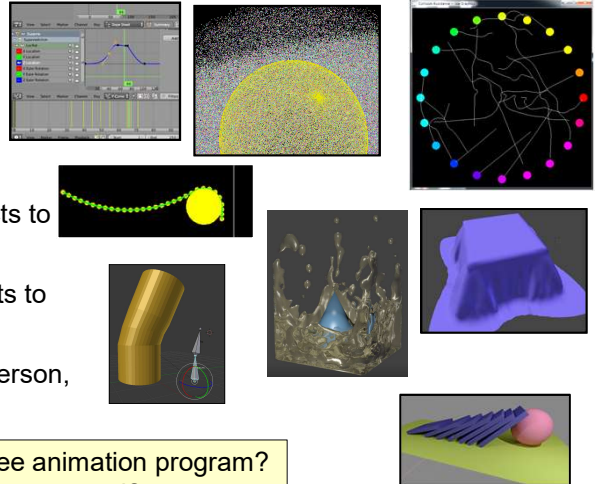
mjb - June 3, 2022



Animation

Rendering is the process of giving motion to your geometric modes. Again, there are questions you need to ask first:

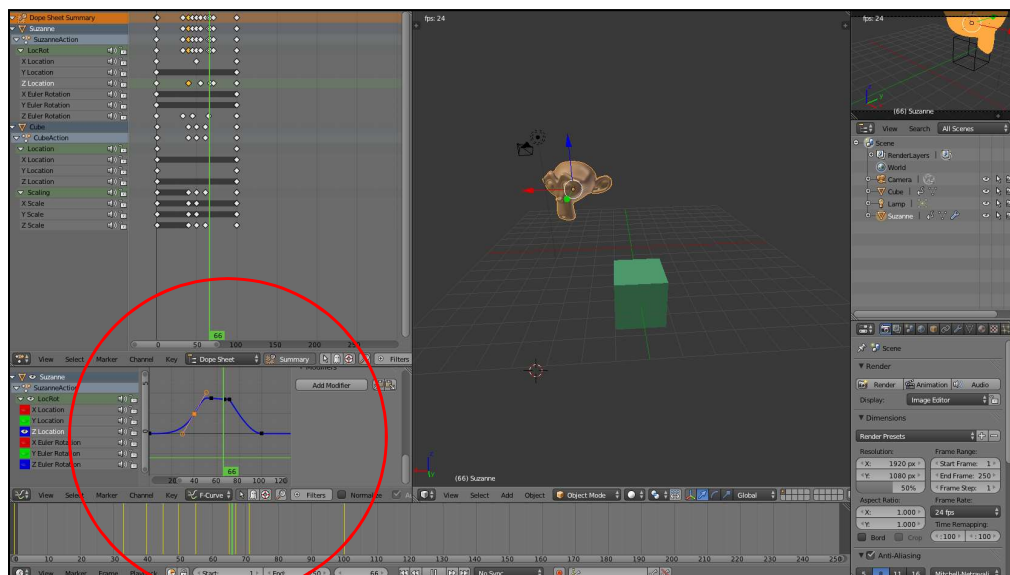
- Why am I doing this?
- Do I want the animation to obey the real laws of physics?
- Am I willing to “fake” the physics to get the objects to *want* to move in a way that I tell it?
- Do I have specific key positions I want the objects to pass through no matter what?
- Do I want to simply record the motion of a real person, animal, etc., and then play it back?



Want to experiment with a free animation program?
Want some notes on how to get started?

<http://cs.oregonstate.edu/~mjb/blender>

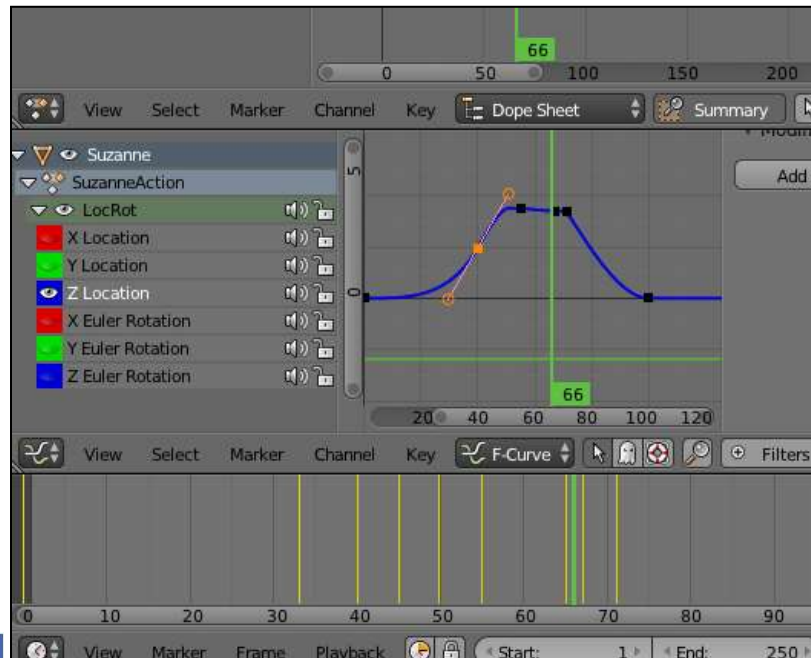
Keyframe Animation



Forcing the geometry to smoothly pass through key positions

Keyframe Animation

63



anim2.mp4

Sidebar: DIY KeyTime Animation

64

A simple C++ class can do it for you

class Keytimes:

```
void AddTimeValue( float time, float value );
float GetFirstTime( );
float GetLastTime( );
int GetNumKeytimes( );
float GetValue( float time );
void PrintTimeValues( );
```

Find this code (keytime.h, keytime.cpp) on:

<http://cs.oregonstate.edu/~mjb/whirlwind>

Keytimes Xpos;

int

main(int argc, char *argv[])

{

 Xpos.AddTimeValue(0.0, 0.000);

 Xpos.AddTimeValue(2.0, 0.333);

 Xpos.AddTimeValue(1.0, 3.142);

 Xpos.AddTimeValue(0.5, 2.718);

 for(float t = 0.; t <= 2.01; t += 0.1) // just to show the example – we don't really use a float in a for-loop

 {

 float v = Xpos.GetValue(t);

 fprintf(stderr, "%8.3ft%8.3f\n", t, v);

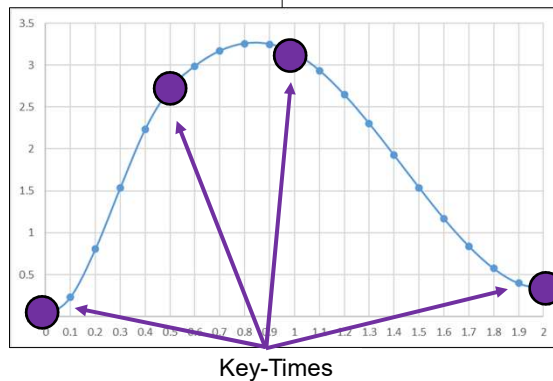
 }

}

(0.00, 0.000)
(0.00, 0.000) (2.00, 0.333)
(0.00, 0.000) (1.00, 3.142) (2.00, 0.333)
(0.00, 0.000) (0.50, 2.718) (1.00, 3.142) (2.00, 0.333)
4 time-value pairs

Time runs from 0.000 to 2.000

0.000	0.000
0.100	0.232
0.200	0.806
0.300	1.535
0.400	2.234
0.500	2.718
0.600	2.989
0.700	3.170
0.800	3.258
0.900	3.250
1.000	3.142
1.100	2.935
1.200	2.646
1.300	2.302
1.400	1.924
1.500	1.539
1.600	1.169
1.700	0.840
1.800	0.574
1.900	0.397
2.000	0.333



Using the System Clock for Timing in Display()

67

```
#define MSEC 10000 // i.e., 10 seconds
Keytimes Xpos, Ypos, Zpos;
Keytimes ThetaX, ThetaY, ThetaZ;

...

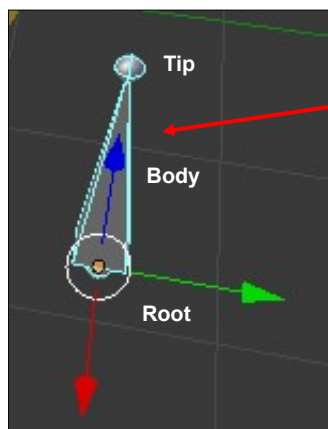
if( AnimationsIsOn )
{
    // # msec into the cycle ( 0 - MSEC-1 ):
    int msec = glutGet( GLUT_ELAPSED_TIME ) % MSEC;

    // turn that into a time in seconds:
    float nowTime = (float)msec / 1000.;
    glPushMatrix( );
    glTranslatef( Xpos.GetValue( nowTime ), Ypos.GetValue( nowTime ), Zpos.GetValue( nowTime ) );
    glRotatef( ThetaX.GetValue( nowTime ), 1., 0., 0. );
    glRotatef( ThetaY.GetValue( nowTime ), 0., 1., 0. );
    glRotatef( ThetaZ.GetValue( nowTime ), 0., 0., 1. );
    << draw the object >>
    glPopMatrix( );
}
```

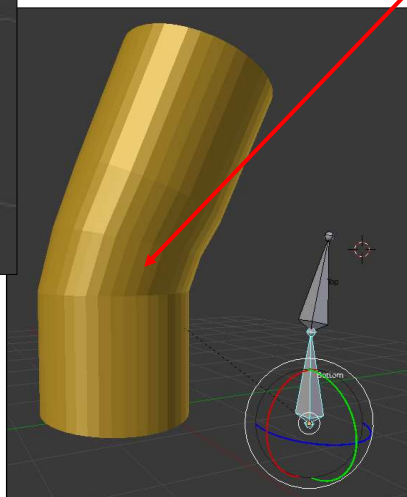
Number of msec in the animation cycle

Rigging Animation

68



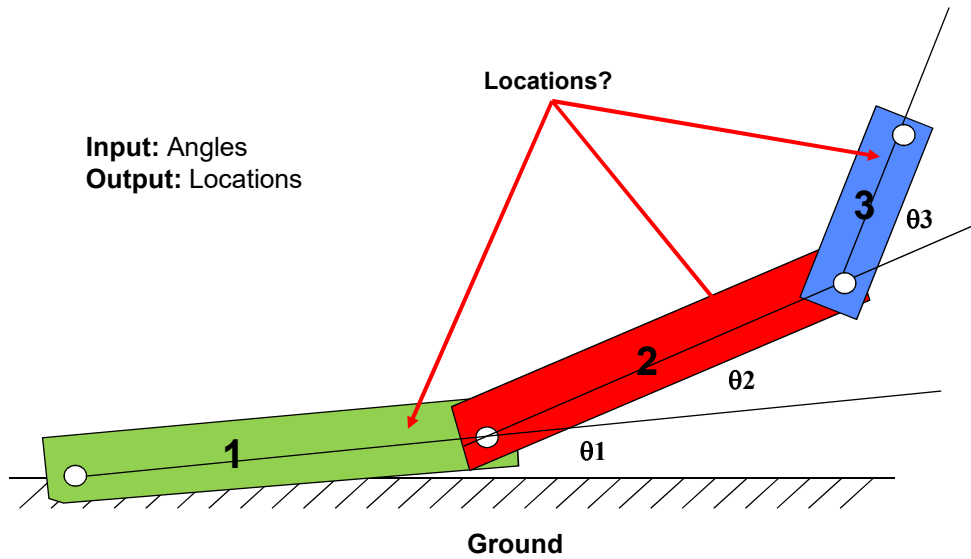
Control the movement of groups of vertices with an armature



Forward Kinematics: Transformation Hierarchies

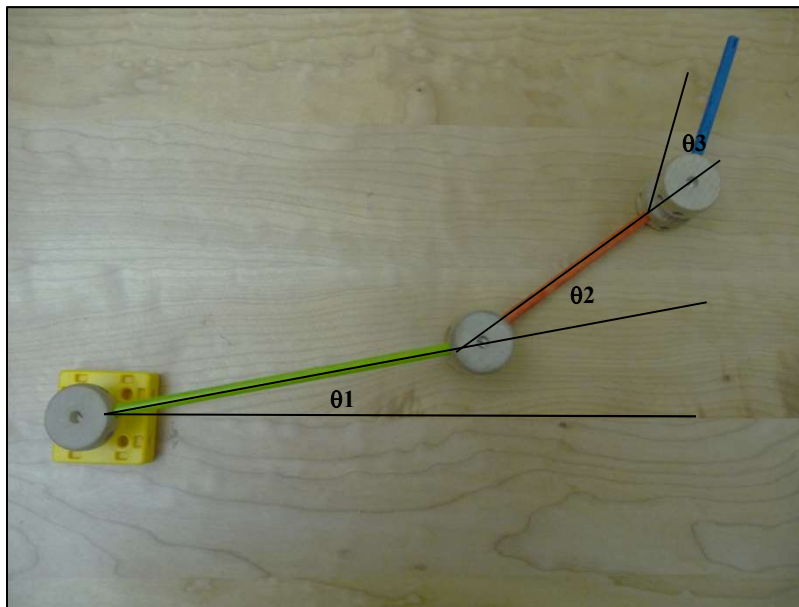
69

Input: Angles
Output: Locations



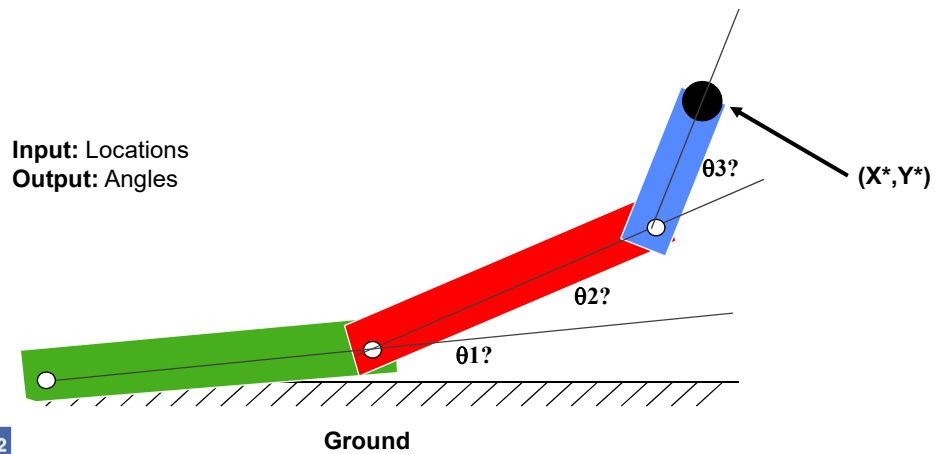
Forward Kinematics: Change Parameters – Things Move (All Children Understand This)

70

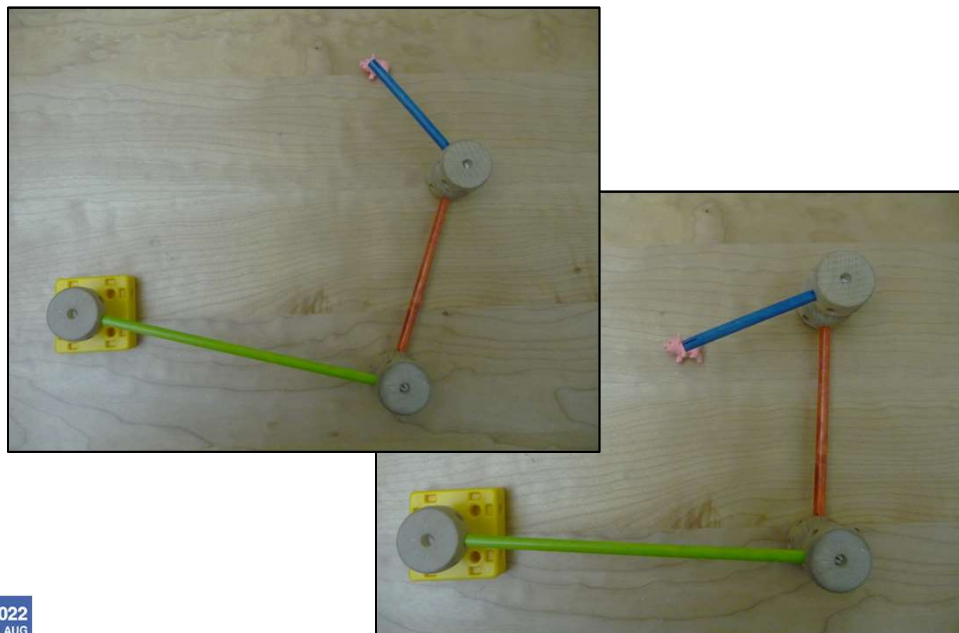


Forward Kinematics solves the problem “if I know the link transformation parameters, where are the links?”.

Inverse Kinematics (IK) solves the problem “If I know where I want the end of the chain to be (X^*, Y^*) , what transformation parameters will put it there?”



Inverse Kinematics (IK): Things Need to Move – What Parameters Will Make Them Do That?



Animating a Human-ish Form

73

Start with this ...

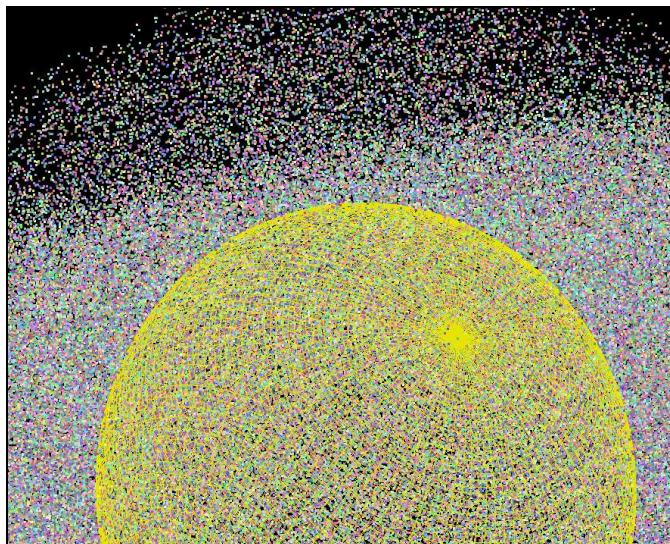


... and turn it into a kinematic model:



Particle Systems: A Cross Between Modeling and Animation?

74



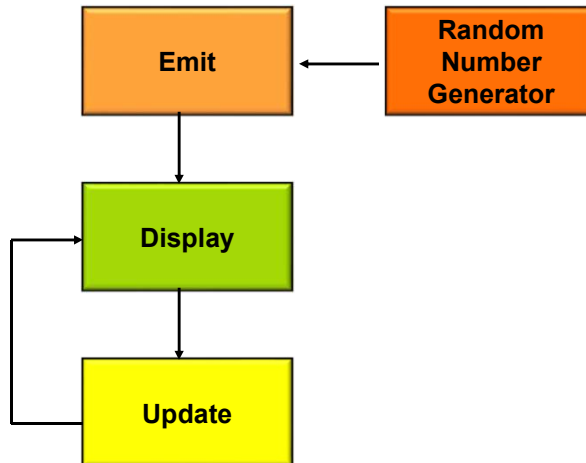
gpu_particles.mp4

Check out this movie! These are particles animated on a GPU.

Particle Systems: A Cross Between Modeling and Animation?

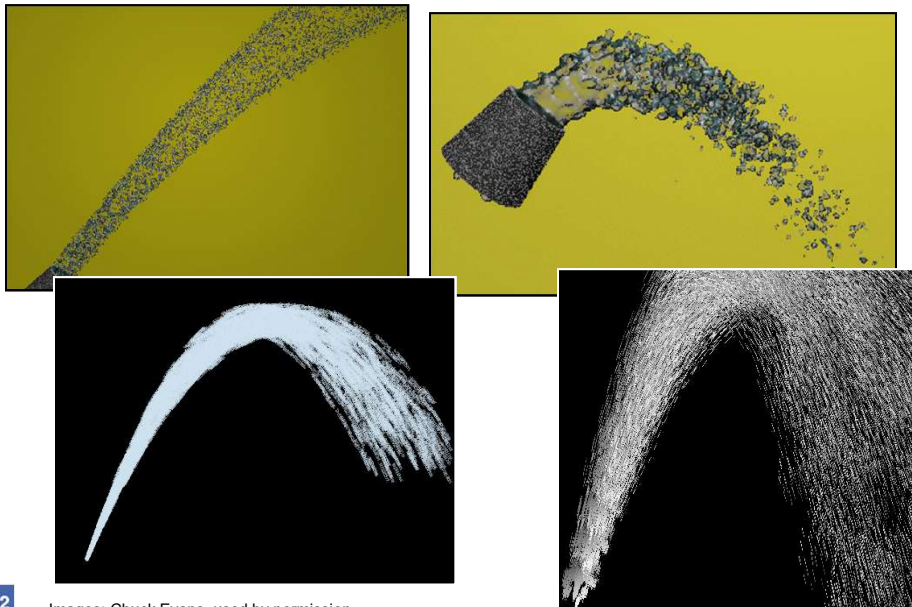
75

The basic process is:



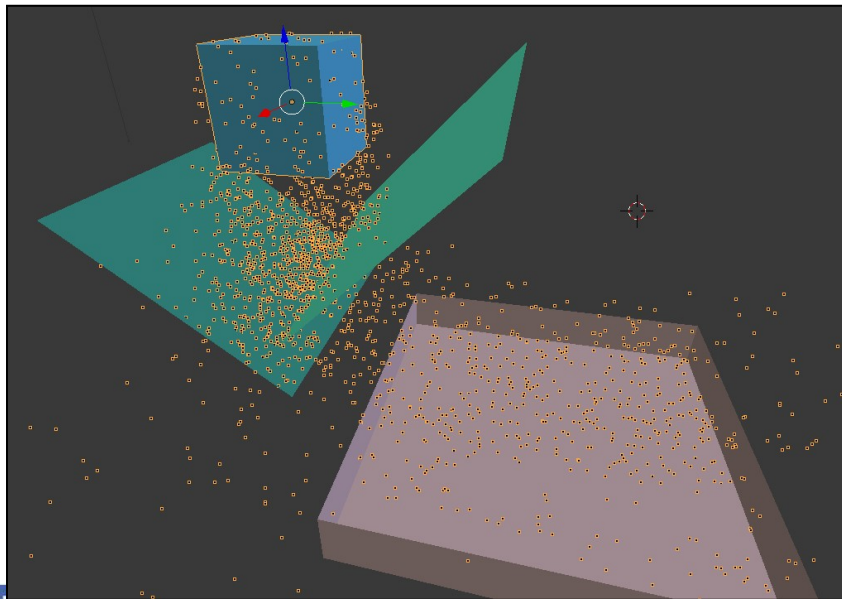
Particle Systems Examples

76



Particle Systems Examples

77



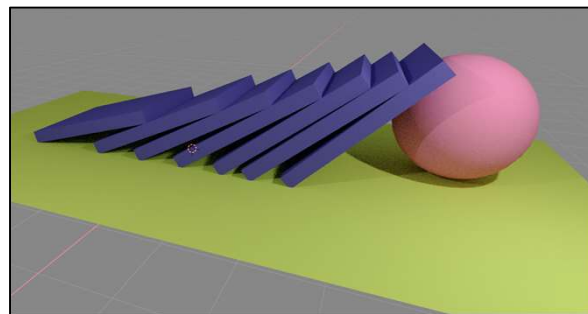
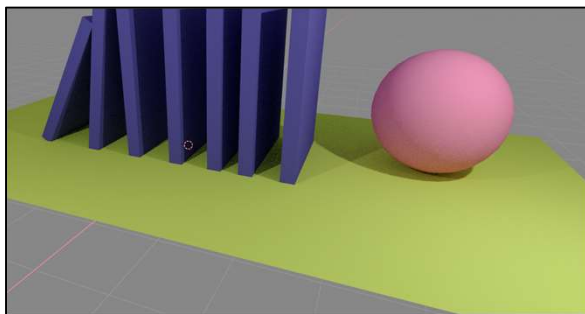
particles.mp4



mjb - June 3, 2022

Animating using Rigid-body Physics

78



Images: Mike Bailey

Newton's second law:

$$\text{force} = \text{mass} * \text{acceleration}$$

or:

$$\text{acceleration} = \{\ddot{x}\} = \text{force} / \text{mass}$$

Newton's Second Law



dominos2.mp4

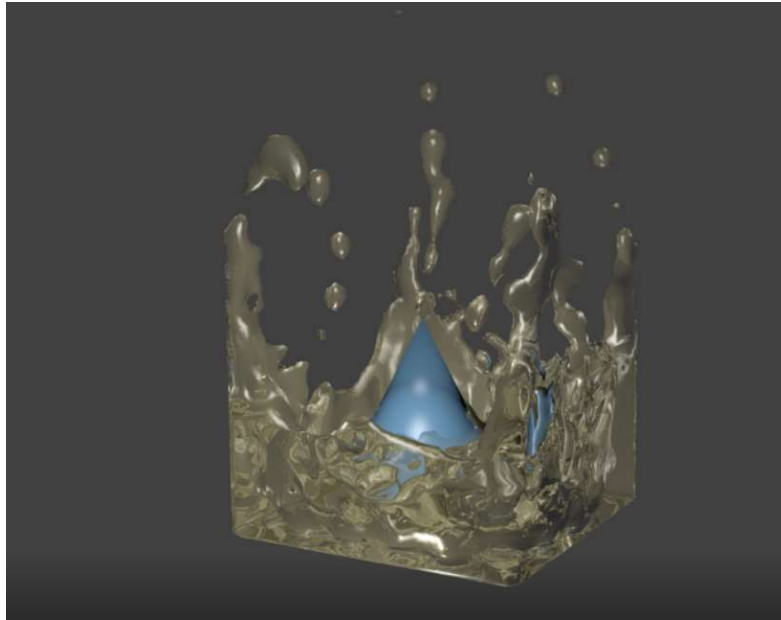
In order to make this work, you need to supply physical properties such as mass, center of mass, moment of inertia, coefficients of friction, coefficients of restitution, etc.



mjb - June 3, 2022

Animating using Fluid Physics

79

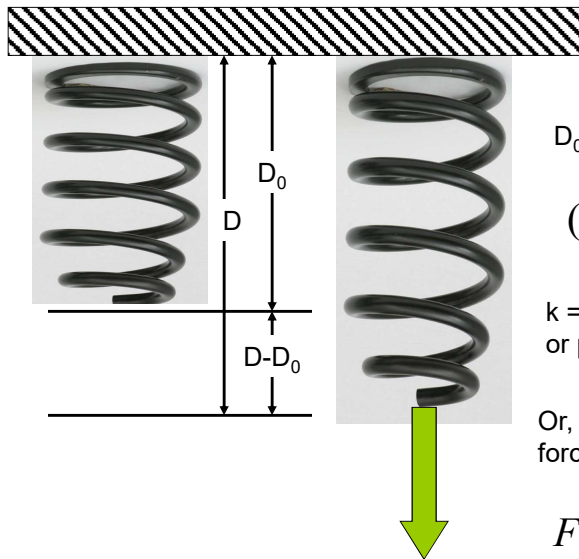


fluid.avi

Image: Mike Bailey

Animating using Spring Physics

80



D_0 = unloaded spring length

$$(D - D_0) = \frac{F}{k}$$

k = **spring stiffness** in Newtons/meter or pounds/inch

Or, if you know the displacement, the force exerted by the spring is:

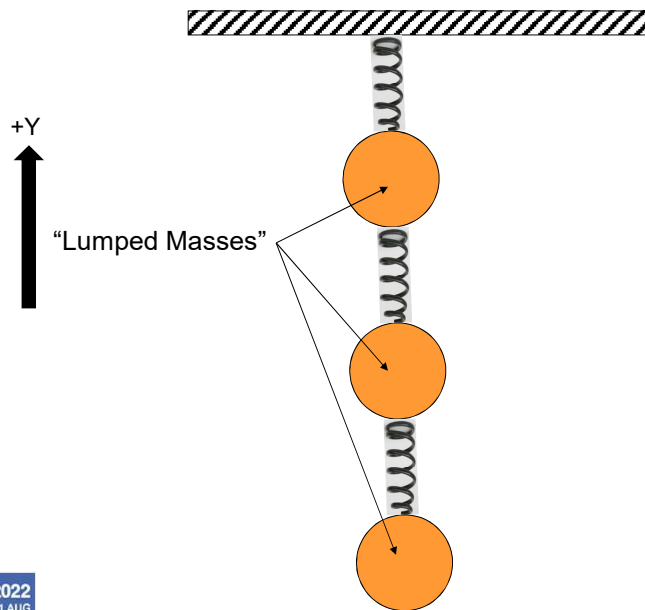
$$F = k(D - D_0)$$

Force = F

This is known as **Hooke's Law**

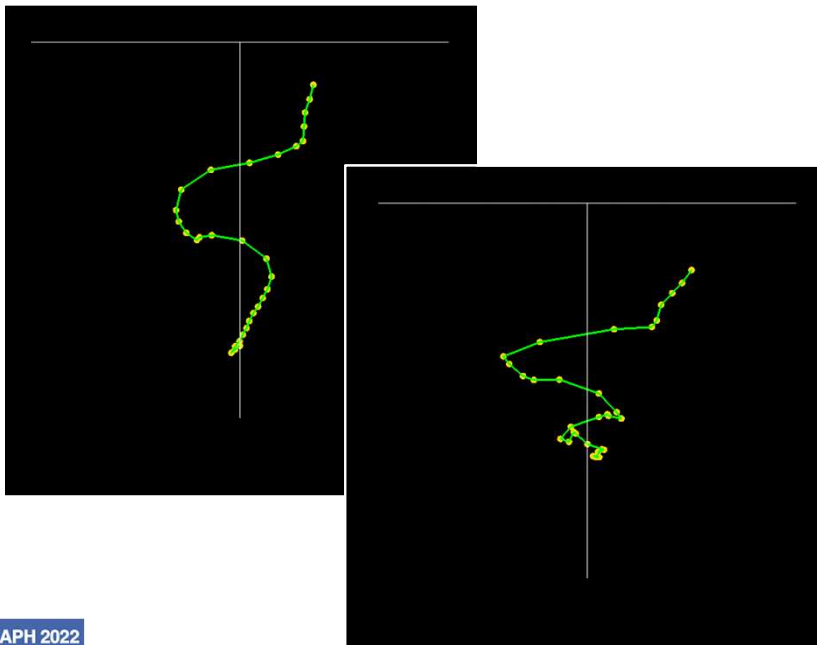
Animating using the Physics of a Mesh of Springs

81



Simulating a Bouncy Chain

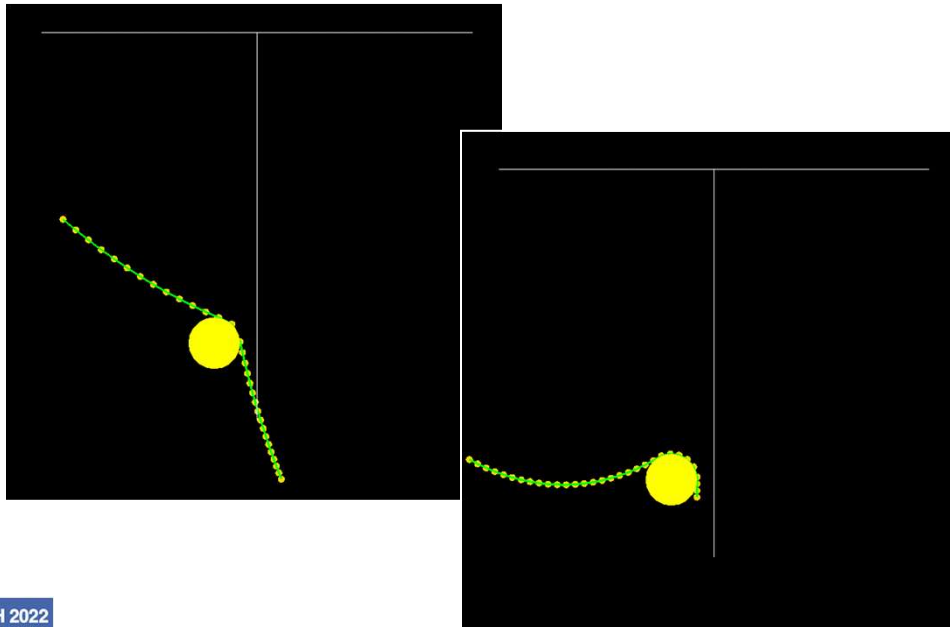
82



chain.mp4

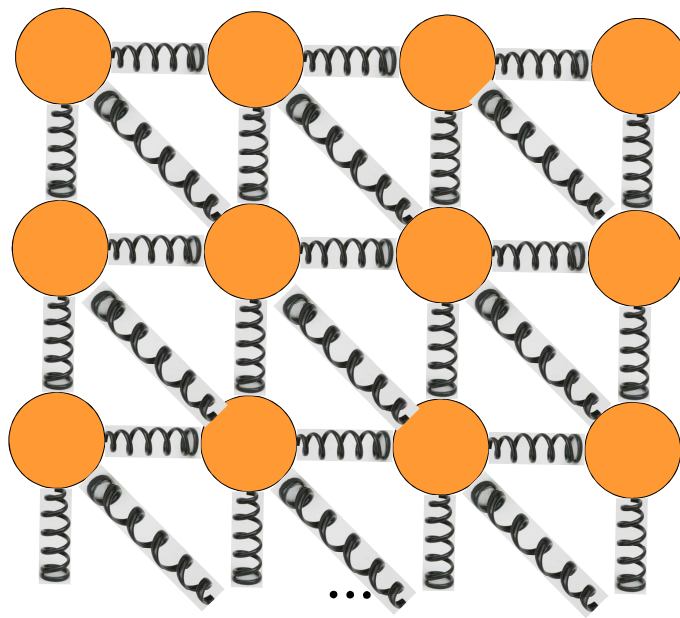
Placing a Physical Barrier in the Scene

83



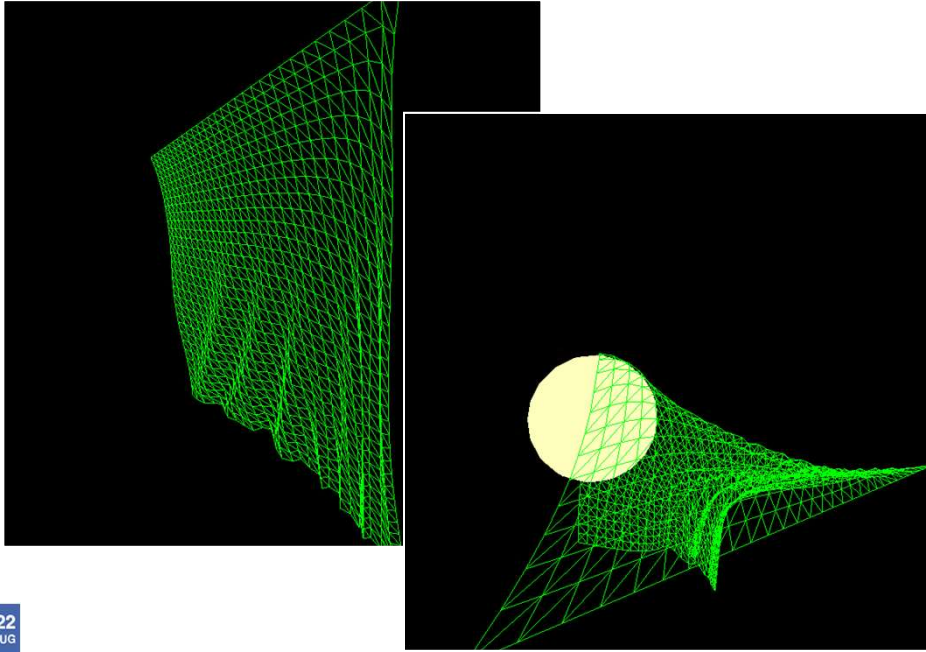
Animating Cloth

84



Cloth Examples

85



mjb - June 3, 2022

Cloth Example

86

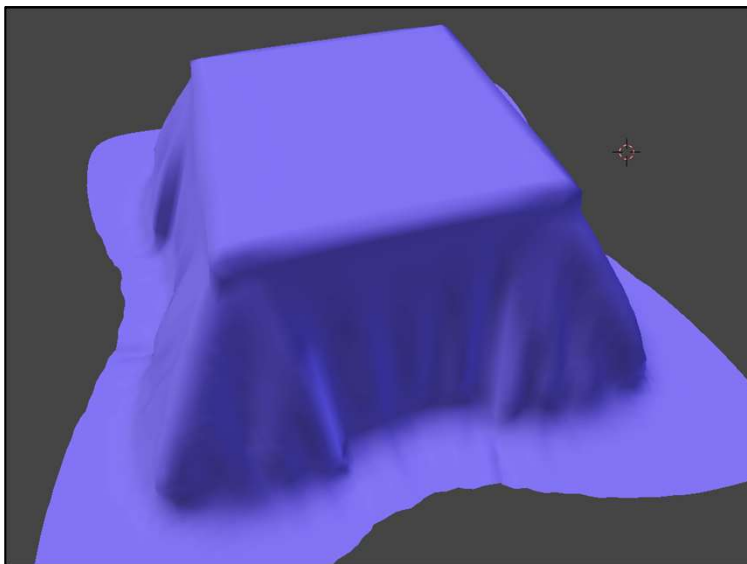


Image: Mike Bailey



cloth.mp4



mjb - June 3, 2022

Functional Animation – “Fake Physics”

87

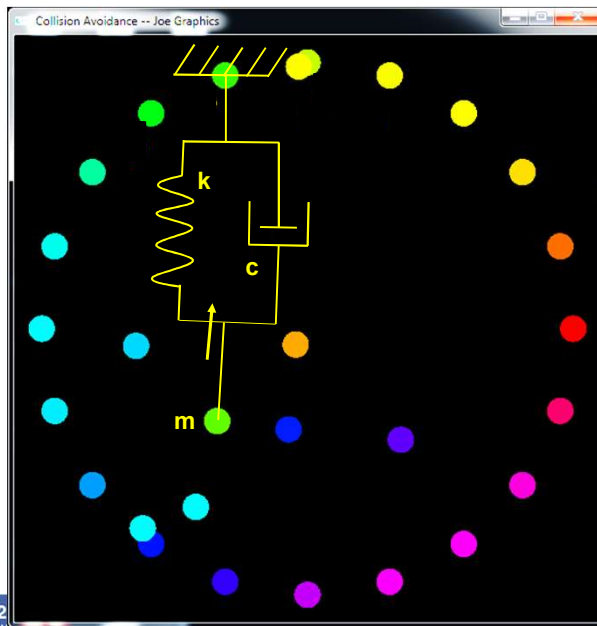


Photo: Mike Bailey

The Challenge: animate a collection of objects, each trying to move to a target, but without colliding with each other

Functional Animation: Make the Object *Want* to Move Towards a Goal Position . . .

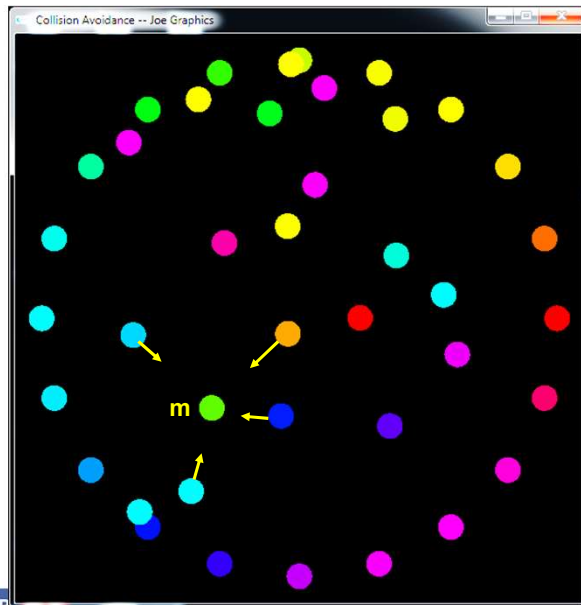
88



$$m\ddot{x} + c\dot{x} + kx = 0$$

Functional Animation:
... While Making it *Want* to Keep Away from all other Objects

89



$$m\ddot{x} = \sum F_{repulsive}$$

$$F_{repulsive} = \frac{C_{repulse}}{d^{Power}}$$

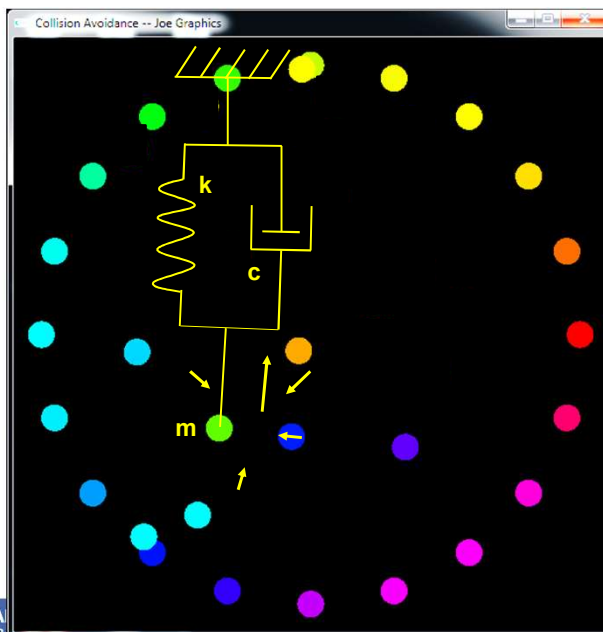
Repulsion Coefficient

Distance between the boundaries of the 2 bodies

Repulsion Exponent

Total Goal – Make the Free Body Move Towards its Final Position
While Being Repelled by the Other Bodies

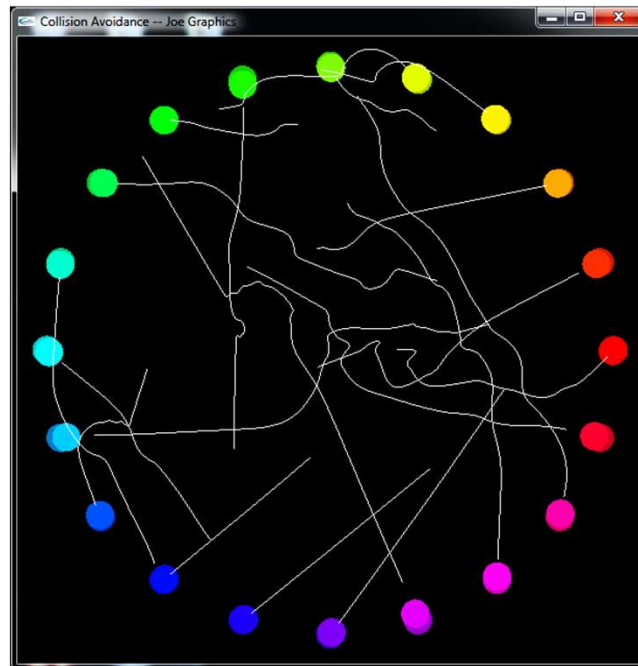
90



$$m\ddot{x} + c\dot{x} + kx = \sum F = \sum F_{repulsive}$$

Functional Animation

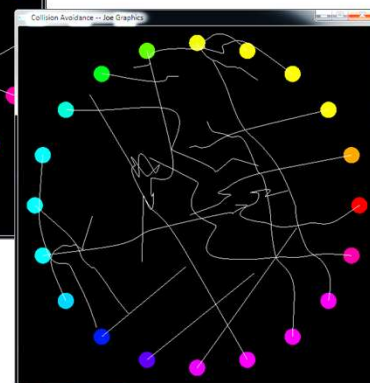
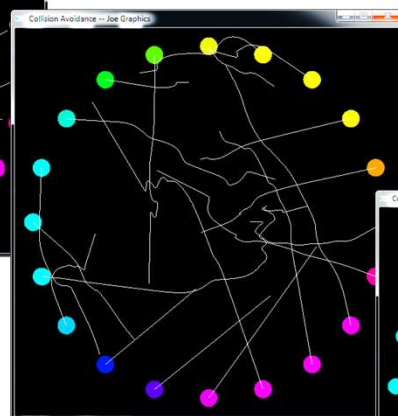
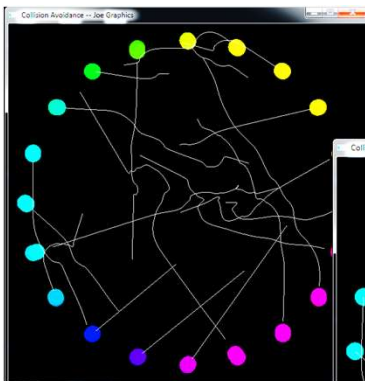
91



avoid.mp4

Increasing the Stiffness

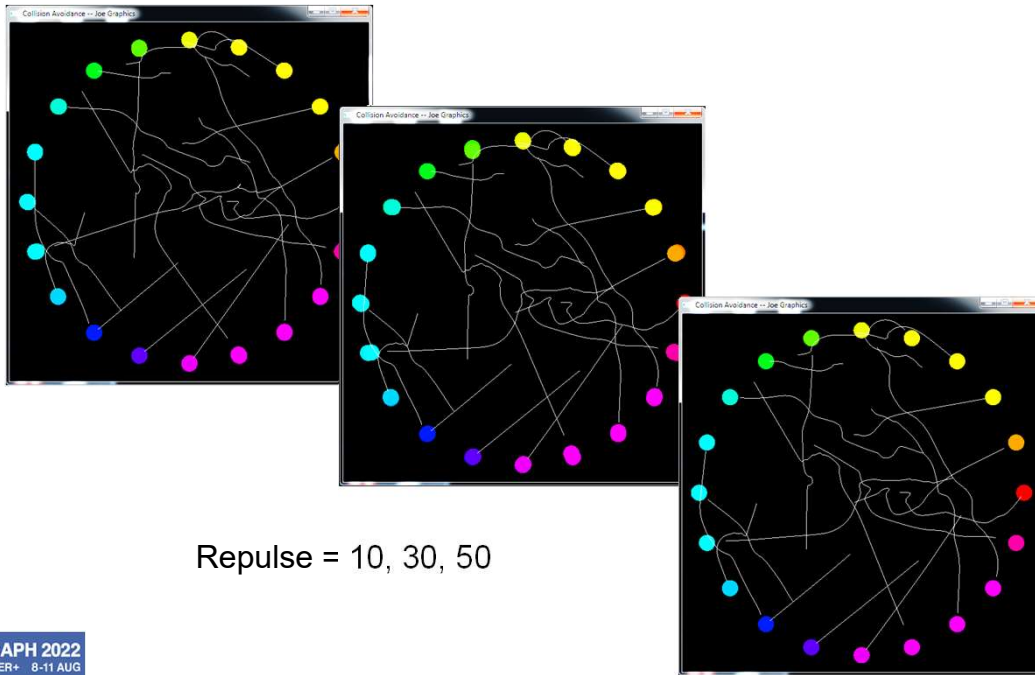
92



Stiffness = 3, 6, 9

Increasing the Repulsion Coefficient

93



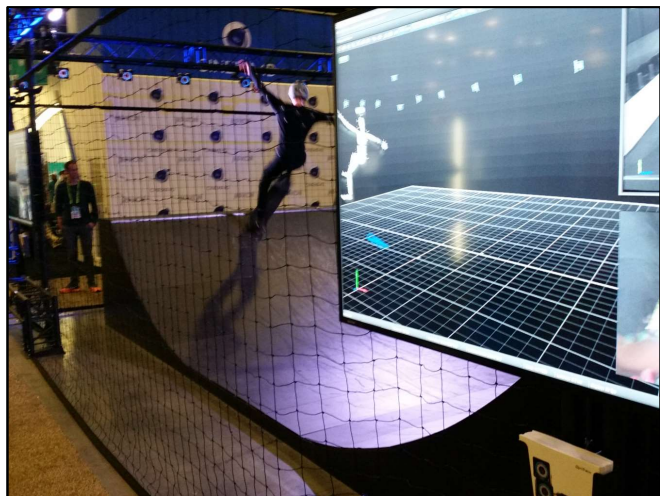
Repulse = 10, 30, 50

Motion Capture ("MoCap") as an Input for Animation

94



Photos: Mike Bailey



Even Animals can be MoCapped

95

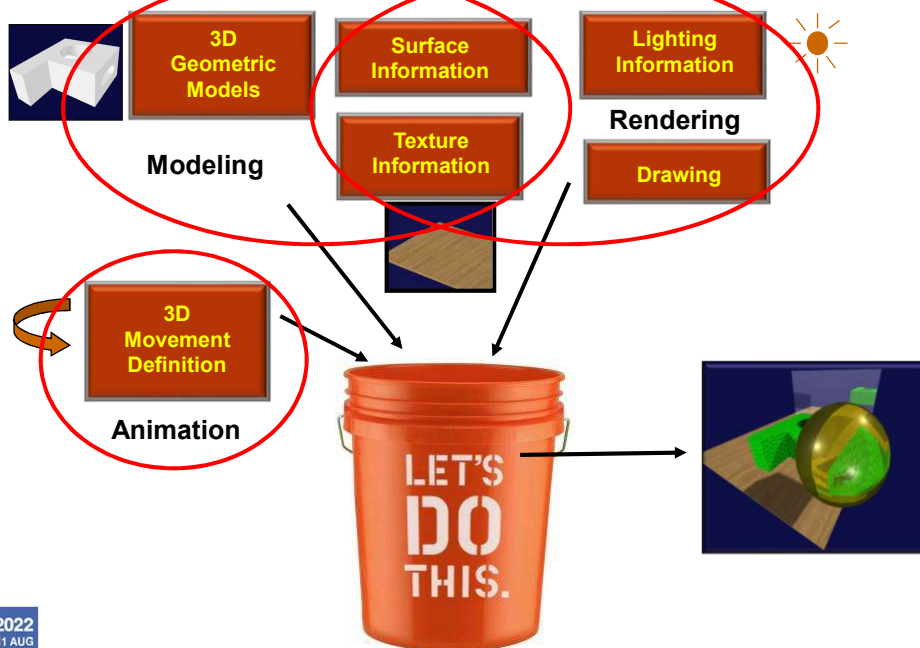


https://www.youtube.com/watch?v=zyq_LQrHpoo

Photo courtesy of: DIGIC Services' Mocap Studio,
used by permission

Summary

96



Conclusions !

97

- SIGGRAPH moments will never come again. Well, this is usually true, but through magic of the 2022 videos, they might. *But, be aware of what is going to be archived and what isn't.* And, if it is to be archived, how long will you have access to it?
- Especially take advantage of the not-to-be-archived or not-to-be-archived-for-very-long events because you cannot re-live them forever.
- Combine what you have just learned here with what else you learn at the conference and *relate them to your career and life goals.*
- Have fun doing it!



98

Where to Find More Information about Computer Graphics and Related Topics

Mike Bailey
Oregon State University

1. References

1.1 General Computer Graphics

SIGGRAPH Online Bibliography Database:

<http://www.siggraph.org/learn/computer-graphics-bibliography-database>

Edward Angel and Dave Shreiner, *Interactive Computer Graphics: A Top-down Approach with OpenGL*, 6th Edition, Addison-Wesley, 2011.

Francis Hill and Stephen Kelley, *Computer Graphics Using OpenGL*, 3rd Edition, Prentice Hall, 2006.

Steve Cunningham, *Computer Graphics: Programming in OpenGL for Visual Communication*, Prentice-Hall, 2007

Alan Watt, *3D Computer Graphics*

<http://cs.oregonstate.edu/~mjb/whirlwind>

Peter Shirley, *Fundamentals of Computer Graphics*, 2nd Edition, AK Peters, 2005.

Andrew Glassner, *Graphics Gems*, Academic Press, 1990.



SIGGRAPH 2022
VANCOUVER+ 8-11 AUG

A Whirlwind Introduction to Computer Graphics

Mike Bailey
mjb@cs.oregonstate.edu

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the Owner/Author.

Copyright 2022 Mike Bailey

SIGGRAPH '22 Educator's Forum, August 07-11, 2022, Vancouver, BC, Canada

ACM 978-1-4503-9366-9/22/08.

10.1145/3532724.3535605

<http://cs.oregonstate.edu/~mjb/whirlwind>

Where to Find More Information about Computer Graphics, Parallel Programming, and Related Topics

Mike Bailey
Oregon State University

1. References

1.1 General Computer Graphics

GraphBib: SIGGRAPH's Online Bibliography Database:

<https://liinwww.ira.uka.de/bibliography/Graphics/siggraph/index.html>

John Kessenich, Graham Sellers, and Dave Shreiner, *OpenGL Programming Guide, 9th Edition*, 2017.

Peter Shirley, *Fundamentals of Computer Graphics*, 5th Edition, AK Peters, 2021.

Edward Angel and Dave Shreiner, *Interactive Computer Graphics: A Top-down Approach with OpenGL*, 6th Edition, Addison-Wesley, 2011.

Graham Sellers, *Vulkan Programming Guide*, Addison-Wesley, 2017.

Sergey Kosarevsky and Viktor Latypov, *3D Graphics Rendering Cookbook*, Packt, 2021.

Francis Hill and Stephen Kelley, *Computer Graphics Using OpenGL*, 3rd Edition, Prentice Hall, 2006.

Steve Cunningham, *Computer Graphics: Programming in OpenGL for Visual Communication*, Prentice-Hall, 2007

Alan Watt, *3D Computer Graphics*, 3rd Edition, Addison-Wesley, 2000.

Andrew Glassner, *Graphics Gems*, Academic Press, 1990.

James Arvo, *Graphics Gems 2*, Academic Press, 1991.

David Kirk, *Graphics Gems 3*, Academic Press, 1992.

Paul Heckbert, *Graphics Gems 4*, Academic Press, 1994.

Alan Paeth, *Graphics Gems 5*, Academic Press, 1995.

Jim Blinn, *A Trip Down the Graphics Pipeline*, Morgan Kaufmann, 1996.

Jim Blinn, *Dirty Pixels*, Morgan Kaufmann, 1998.

David Rogers, *Procedural Elements for Computer Graphics*, McGraw-Hill, 1997.

1.2 Vulkan

<http://cs.oregonstate.edu/~mjb/vulkan>

Graham Sellers, *Vulkan Programming Guide*, Addison-Wesley, 2017.

Pawel Lapinski, *Vulkan Cookbook*, Packt Publishing, 2017.

Sergey Kosarevsky and Viktor Latypov, *3D Graphics Rendering Cookbook*, Packt, 2021.

Kenwright, *Introduction to Computer Graphics and the Vulkan API*, 3rd Edition, 2018.

1.3 Math and Geometry

Ron Goldman, *An Integrated Introduction to Computer Graphics and Geometric Modeling*, CRC Press, 2009.

Michael Mortenseon, *Geometric Transformations for 3D Modeling*, 2nd Edition, Industrial press, 2007.

Michael Mortenson, *Geometric Modeling*, John Wiley & Sons, 2006.

Eric Lengyel, *Mathematics for 3D Game Programming and Computer Graphics*, Charles River Media, 2002.

Jean Gallier, *Curves and Surfaces in Geometric Modeling*, Morgan Kaufmann, 2000.

Walter Taylor, *The Geometry of Computer Graphics*, Wadsworth & Brooks/Cole, 1992.

Gerald Farin, *Curves and Surfaces for Computer Aided Geometric Design*, 3rd Edition, Academic Press, 2001.

Gerald Farin and Dianne Hansford, *The Geometry Toolbox for Graphics and Modeling*, AK Peters, 1998.

Joe Warren and Henrik Weimer, *Subdivision Methods for Geometric Design: A Constructive Approach*, Morgan Kaufmann, 2001.

Barrett O’Neil, *Elementary Differential Geometry*, Academic Press, 1997.

Joseph O’Rourke, *Computational Geometry in C*, Cambridge University Press, 1996.

Christopher Hoffman, *Geometric & Solid Modeling*, Morgan Kaufmann, 1989.

I.D. Faux and M.J. Pratt, *Computational Geometry for Design and Manufacture*, Ellis-Horwood, 1979.

Eric Stollnitz, Tony DeRose, and David Salesin, *Wavelets for Computer Graphics*, Morgan-Kaufmann,

1996.

Ronen Barzel, *Physically-Based Modeling for Computer Graphics*, Academic Press, 1992.

David Rogers and J. Alan Adams, *Mathematical Elements for Computer Graphics*, McGraw-Hill, 1989.

John Snyder, *Generative Modeling for Computer Graphics and Computer Aided Design*, Academic Press, 1992.

1.4 Scientific Visualization

Tamara Munzner, *Visualization Analysis & Design*, A.K. Peters, 2015.

John Dill, Rae Earnshaw, David Kasik, John Vince, and Pak Chung Wong, *Expanding the Frontiers of Visual Analytics and Visualization*, Springer, 2012.

Christopher Johnson and Charles Hansen, *The Visualization Handbook*, Elsevier Academic Press, 2005.

Klaus Engel, Markus Hadwiger, Joe Kniss, Christof Rezk-Salama, and Daniel Weiskopf, *Real-Time Volume Graphics*, A.K. Peters, 2006.

David Thompson, Jeff Braun, and Ray Ford, *OpenDX: Paths to Visualization*, Visualization and Imagery Solutions, Inc., 2001.

Chandrajit Bajaj, *Data Visualization Techniques*, John Wiley & Sons, 1999.

Min Chen, Arie Kaufman, and Roni Yagel, *Volume Graphics*, Springer-Verlag, 2000.

William Schroeder, Ken Martin, and Bill Lorensen, *The Visualization Toolkit*, 3rd Edition, Prentice-Hall, 2004.

Luis Ibanez and William Schroeder, *The ITK Software Guide: The Insight Segmentation and Registration Toolkit (version 1.4)*, Prentice-Hall, 2003.

Greg Nielson, Hans Hagen, and Heinrich Müller, *Scientific Visualization: Overviews, Methodologies, Techniques*, IEEE Computer Society Press, 1997.

Brand Fortner, *The Data Handbook: A Guide to Understanding the Organization and Visualization of Technical Data*, Spyglass, 1992.

William Kaufmann and Larry Smarr, *Supercomputing and the Transformation of Science*, Scientific American Library, 1993.

Robert Wolff and Larry Yaeger, *Visualization of Natural Phenomena*, Springer-Verlag, 1993.

Peter Keller and Mary Keller, *Visual Cues: Practical Data Visualization*, IEEE Press, 1993.

1.5 Shaders

Mike Bailey and Steve Cunningham, *Computer Graphics Shaders: Theory and Practice*, Second Edition, CRC Press, 2012.

John Kessenich, Graham Sellers, and Dave Shreiner, *OpenGL Programming Guide, 9th Edition*, 2017.

Randi Rost, Bill Licea-Kane, Dan Ginsburg, John Kessenich, Barthold Lichtenbelt, Hugh Malan, and Mike Weiblen, *OpenGL Shading Language*, Addison-Wesley, 2009. (3rd Edition)

Steve Upstill, *The RenderMan Companion*, Addison-Wesley, 1990.

Tony Apodaca and Larry Gritz, *Advanced RenderMan: Creating CGI for Motion Pictures*, Morgan Kaufmann, 1999.

Saty Raghavachary, *Rendering for Beginners: Image Synthesis using RenderMan*, Focal Press, 2005.

Randima Fernando, *GPU Gems*, NVIDIA, 2004.

Matt Pharr, Randima Fernando, *GPU Gems 2*, NVIDIA, 2005.

Hubert Nguyen, *GPU Gems 3*, NVIDIA, 2007.

<http://www.clockworkcoders.com/oglsl>

1.6 Gaming

Jesse Schell, *The Art of Game Design*, Morgan-Kaufmann, 3rd Edition, 2019.

David Hodgson, Bryan Stratten, and Alice Rush, *Paid to Play: An Insider's Guide to Video Game Careers*, Prima, 2006.

Alan Watt and Fabio Polcarpo, *Advanced Game Development with Programmable Graphics Hardware*, AK Peters, 2005.

Jacob Habgood and Mark Overmars, *The Game Maker's Apprentice*, Apress, 2006.

David Eberly, *3D Game Engine Design: A Practical Approach to Real-Time Computer Graphics*, Morgan Kaufmann, 2006.

Alan Watt and Fabio Polcarpo, *3D Games: Real-time Rendering and Software Technology*, Addison-Wesley, 2001.

Eric Lengyel, *Mathematics for 3D Game Programming and Computer Graphics*, Charles River Media, 2002.

David Bourg, *Physics for Game Developers*, O'Reilly and Associates, 2002.

Munlo Coutinho, *Dynamic Simulations of Multibody Systems*, Springer Verlag, 2001.

Mark DeLoura, *Game Programming Gems*, Charles River Media, 2000.

Mark DeLoura, *Game Programming Gems 2*, Charles River Media, 2001.

Dante Treglia, *Game Programming Gems 3*, Charles River Media, 2002.

Andrew Kimse, *Game Programming Gems 4*, Charles River Media, 2004.

Kim Pallister, *Game Programming Gems 5*, Charles River Media, 2005.

Mike Dickheiser, *Game Programming Gems 6*, Charles River Media, 2006.

Scott Jacobs, *Game Programming Gems 7*, Charles River Media, 2008.

Adam Lake, *Game Programming Gems 8*, Charles River Media, 2010.

<http://www.gamedev.net>

<http://www.gamasutra.net>

<http://www.yoyogames.com>

1.7 Color and Perception

Theresa-Marie Rhyne, *Applying Color Theory to Digital Media and Visualization*, CRC Press, 2017.

Maureen Stone, *A Field Guide to Digital Color*, AK Peters, 2003.

Roy Hall, *Illumination and Color in Computer Generated Imagery*, Springer-Verlag, 1989.

David Travis, *Effective Color Displays*, Academic Press, 1991.

L.G. Thorell and W.J. Smith, *Using Computer Color Effectively*, Prentice Hall, 1990.

Edward Tufte, *The Visual Display of Quantitative Information*, Graphics Press, 1983.

Edward Tufte, *Envisioning Information*, Graphics Press, 1990.

Edward Tufte, *Visual Explanations*, Graphics Press, 1997.

Edward Tufte, *Beautiful Evidence*, Graphics Press, 2006.

Howard Resnikoff, *The Illusion of Reality*, Springer-Verlag, 1989.

1.8 Rendering

Sergey Kosarevsky and Viktor Latypov, *3D Graphics Rendering Cookbook*, Packt Publishing, 2021.

Tomas Akenine-Möller, Eric Haines, Naty Hoffman, Angelo Pesce, Michal Iwanicki, and Sébastien Hillaire, *Real-time Rendering*, CRC Press, 2018.

Andrew Glassner, *Principles of Digital Image Synthesis*, Morgan Kaufmann, 1995.

Michael Cohen and John Wallace, *Radiosity and Realistic Image Synthesis*, Morgan-Kaufmann, 1993.

Andrew Glassner, *An Introduction to Ray Tracing*, Academic Press, 1989.

Rosalee Wolfe, *3D Graphics: A Visual Approach*, Oxford Press, 1999.

Ken Joy et al, *Image Synthesis*, IEEE Computer Society Press, 1988.

1.9 Images

David Ebert et al, *Texturing and Modeling*, 2nd Edition, Academic Press, 1998.

Alan Watt and Fabio Polcarpo, *The Computer Image*, Addison-Wesley, 1998.

Ron Brinkman, *The Art and Science of Digital Compositing*, Morgan Kaufmann, 1999.

John Miano, *Compressed Image File Formats*, Addison-Wesley, 1999.

1.10 Animation

Alan Watt and Mark Watt, *Advanced Animation and Rendering Techniques*, Addison-Wesley, 1998.

Nadia Magnenat Thalmann and Daniel Thalmann, *Interactive Computer Animation*, Prentice-Hall, 1996.

Philip Hayward and Tana Wollen, *Future Visions: New Technologies of the Screen*, Indiana University Press, 1993.

1.11 Virtual Reality

John Vince, *Virtual Reality Systems*, Addison-Wesley, 1995.

1.12 Web

Kouichi Matsuda and Rodger Lea, *WebGL Programming Guide*, Addison Wesley, 2015.

Tony Parisi, *Programming 3D Applications with HTML5 and WebGL: 3D Animation and Visualization for Web Pages*, O'Reilly, 2014.

Tony Parisi, *WebGL: Up and Running*, O'Reilly, 2012.

Don Brutzman and Leonard Daly, *X3D: Extensible 3D Graphics for Web Authors*, Morgan Kaufmann, 2007

Rémi Arnaud and Mark Barnes, *Collada – Sailing the Gulf of 3D Digital Content Creation*, AK Peters, 2006.

Gene Davis, *Learning Java Bindings for OpenGL (JOGL)*, AuthorHouse, 2004.

Andrea Ames, David Nadeau, John Moreland, *The VRML 2.0 Sourcebook*, John Wiley & Sons, 1997.

Bruce Eckel, *Thinking in Java*, Prentice-Hall, 1998.

David Flanagan, *Java in a Nutshell*, O'Reilly & Associates, 5th edition, 2005.

David Flanagan, *Java Examples in a Nutshell*, O'Reilly & Associates, 3rd edition, 2004.

Rasmus Lerdorf and Kevin Tatroe, *Programming PHP*, O'Reilly, 2002.

Yukihiro Matsumoto, *Ruby in a Nutshell*, O'Reilly, 2003.

1.13 Stereographics

David McAllister, *Stereo Computer Graphics and Other True 3D Technologies*, Princeton University Press, 1993.

Lenny Lipton, *The CrystalEyes Handbook*, StereoGraphics Corporation, 1991.

Shab Levy, *Stereoscopic Imaging: A Practical Guide*, Gravitrax Creations, 2008.

1.14 Graphics Miscellaneous

John Blain, *The Complete Guide to Blender Graphics: Computer Modeling and Animation*, 7th Edition, CRC Press, 2022.

Ed Catmull, *Creativity, Inc: Overcoming the Unseen Forces that Stand in the Way of True Inspiration*, Random House, 2014.

David Price, *The Pixar Touch: The Making of a Company*, Vintage Books, 2009.

Alvy Ray Smith, *A Biography of the Pixel*, MIT Press, 2021.

Jacob Gaboury, *Image Objects: An Archeology of Computer Graphics*, MIT Press, 2021.

Andrew Glassner, *Deep Learning: From Basics to Practice: Volumes I and II*, The Imaginary Institute, 2018.

Jason van Gumster, *Blender for Dummies*, Fourth Edition, Wiley, 2020.

Richard S. Wright, Nicholas Haemel, Graham Sellers, and Benjamin Lipchak *OpenGL SuperBible*, 5th Edition, Pearson, 2011.

Aaftab Munshi, Dan Ginsburg, and Dave Shreiner, *OpenGL ES 2.0*, Addison-Wesley, 2008.

Tom McReynolds and David Blythe, *Advanced Graphics Programming Using OpenGL*, Morgan Kaufmann, 2005.

Edward Angel, *OpenGL: A Primer*, Addison-Wesley, 2009.

Andrew Glassner, *Recreational Computer Graphics*, Morgan Kaufmann, 1999.

Anne Spalter, *The Computer in the Visual Arts*, Addison-Wesley, 1999.

Jef Raskin, *The Humane Interface*, Addison-Wesley, 2000.

Ben Shneiderman, *Designing the User Interface*, Addison-Wesley, 1997.

Clark Dodsworth, *Digital Illusion*, Addison-Wesley, 1997.

Isaac Victor Kerlow, *The Art of 3-D: Computer Animation and Imaging*, 2000.

Isaac Victor Kerlow and Judson Rosebush, *Computer Graphics for Designers and Artists*, Van Nostrand Reinhold, 1986.

Mehmed Kantardzic, *Data Mining: Concepts, Models, Methods, and Algorithms*, Wiley, 2003.

William Press, Saul Teukolsky, William Vetterling, and Brian Flannery, *Numerical Recipes in C*, Cambridge University Press, 1997.

James Skakoon and W. J. King, *The Unwritten Laws of Engineering*, ASME Press, 2001.

1.15 Parallel Programming

James Reinders, Ben Ashbaugh, James Brodman, Michael Kinser, John Pennycook, and Xinmin Tian, *Data Parallel C++*, Intel Corporation, 2021.

Michael Voss, Rafael Asenjo, and James Reinders, *Pro TBB: C++ Parallel Programming with Threading Building Blocks*, aPress Open, 2019.

Michael McCool, Arch Robinson, and James Reinders, *Structured Parallel Programming*, Morgan Kaufmann, 2012.

Jo Van Hoey, *Beginning x64 Assembly Programming*, Apress, 2019.

Jim Jeffers and James Reinders, *Intel Xeon Phi Coprocessor High-Performance Programming*, Morgan-Kaufmann, 2013.

Peter Pacheco, *An Introduction to Parallel Programming*, Morgan-Kaufmann, 2011.

James Reinders and Jim Jeffers, *High Performance Parallelism Pearls*, Morgan Kaufmann, 2015.

Aaftah Munshi, Benedict Gaster, Timothy Mattson, James Fung, and Dan Ginsburg, *OpenCL*

- Programming Guide* Addison-Wesley, 2012.
- Benedict Gaster, Lee Howes, David Kaeli, Perhaad Mistry, and Dana Schaa, *Heterogeneous Computing with OpenCL*, Morgan-Kaufmann, 2012.
- Wen-mei Hwu, *GPU Computing Gems I*, Morgan-Kaufmann, 2011.
- Wen-mei Hwu, *GPU Computing Gems II*, Morgan-Kaufmann, 2011.
- David Kirk, Wen-mei Hwu, *Programming Massively Parallel Processors: A Hands-on Approach*, Morgan-Kaufmann, 2010.
- Maurice Herlihy and Nir Shavit, *The Art of Multiprocessor Programming*, Morgan Kaufmann, 2008.
- Rohit Chandra, Leonardo Dagun, Dave Kohr, Dror Maydan, Jeff McDonald, Ramesh Menon, *Parallel Programming in OpenMP*, Morgan Kaufmann, 2001.
- Bradford Nichols, Dick Buttlar, and Jacqueline Proudix Farrell, *Pthreads Programming*, O'Reilly, 1998.
- Ian Foster, *Designing and Building Parallel Programs*, Addison-Wesley, 1995.

2. Periodicals

- Computer Graphics and Applications*: published by IEEE
(<http://www.computer.org> , 714-821-8380)
- Computer Graphics World*
(<http://www.cgw.com> , 603-891-0123)
- Journal of Computer Graphics Techniques*
(<http://jcgt.org>)
- Computer Graphics Quarterly*: published by ACM SIGGRAPH
(<http://www.siggraph.org> , 212-869-7440)
- Computer Graphics Forum*: published by Eurographics
(<https://www.eurographics.org/publications/cgf/>)
- Computers & Graphics*, published by Elsevier
(<http://www.elsevier.com/locate/cag>)
- Transactions on Visualization and Computer Graphics*: published by IEEE
(<https://www.computer.org/csdl/journal/tg> , 714-821-8380)
- Transactions on Graphics*: published by ACM
(<https://dl.acm.org/journal/tog> , 212-869-7440)
- ~~*Cinefex*~~
(~~<http://www.cinefex.com> , 951-781-1917~~)

3. Professional organizations

- ACMAssociation for Computing Machinery
<http://www.acm.org>
212-869-7440
- SIGGRAPHACM Special Interest Group on Computer Graphics
<http://www.siggraph.org>
212-869-7440
- SIGCHIACM Special Interest Group on Computer-Human Interfaces
<http://www.acm.org/sigchi>
212-869-7440
- SIGHPCACM Special Interest Group on High-Performance Computing
<http://sighpc.org>
212-869-7440
- EuroGraphics ...European Association for Computer Graphics
<http://www.eg.org>
Fax: +41-22-757-0318
- IEEE.....Institute of Electrical and Electronic Engineers
<http://www.computer.org>
202-371-0101
- IGDAInternational Game Developers Association
<http://www.igda.org>
856-423-2990
- NAB.....National Association of Broadcasters
<http://www.nab.org>
800-521-8624
- ASMEAmerican Society of Mechanical Engineers
<http://www.asme.org>
800-THE-ASME

4. Upcoming Conferences

- ACM SIGGRAPH:
2022: August 7-11, Vancouver, BC (will be both in-person and online)
<http://s2022.siggraph.org>
- ACM SIGGRAPH Asia:
2022: December 6-9, Daegu, South Korea
<https://sa2022.siggraph.org/en/>
2023: December 12-15 – Sydney, Australia
<https://sa2023.siggraph.org/en/>

ACM SIGCHI:

2023: April 23-28 —Hamburg, Germany
<https://chi2023.acm.org>

SC: International Conference for High Performance Computing, Networking, Storage, and Analysis:

2022: November 13-18 – Dallas, TX
<https://sc22.supercomputing.org>

OpenMPCon:

2023: ??? – ???
<http://openmpcon.org>

IEEE Visualization:

2022: October 16-21 – Oklahoma City, OK
<http://ieeevis.org>

Eurographics

2022: April 25-29 – Reims, France
<https://eg2022.univ-reims.fr/>

Game Developers Conference:

2023: March 20-24 – San Francisco, CA
<http://www.gdconf.com>

E3Expo

2022: “Summer” – Los Angeles, CA
<http://www.e3expo.com>

PAX West (Penny Arcade Expo)

2022: September 2-5 – Seattle, WA
<https://west.paxsite.com>

ASME International Design Engineering Technical Conferences (includes the Computers and Information in Engineering sub-conference):

2022: August 14-17 – St. Louis, MO
<https://event.asme.org/idetc-cie/program>

National Association of Broadcasters (NAB):

2023: April 16-19 -- Las Vegas, NV
<https://nabshow.com/2023>